# Apuntes Acceso a Datos – UT1: Ficheros y formatos de intercambio de datos

Módulo de Acceso a Datos · Tanda 1: guía didáctica, mapa de contenidos y tema 1.1

# Guía didáctica de la unidad

## 1. Contenidos de la unidad

1. Repaso de ficheros secuenciales y aleatorios, directorios, flujos de bytes y caracteres, manejo de excepciones.
2. Serialización y deserialización de objetos.
3. Ficheros de intercambio de datos: **JSON** y **XML**.
4. Analizadores sintácticos (**DOM, SAX, binding**).
5. Consultas sobre XML con **XPath**.
6. Conversión entre formatos (**JSON ↔ XML**).

## 2. Resultado de aprendizaje y criterios de evaluación

**RA1:** desarrolla aplicaciones que gestionan información almacenada en ficheros, identificando el campo de aplicación de los mismos y utilizando clases específicas.

| CE | Criterio |
|---|---|
| a | Se han utilizado clases para la gestión de ficheros y directorios |
| b | Se han valorado las ventajas y los inconvenientes de las distintas formas de acceso |
| c | Se han utilizado clases para recuperar información almacenada en ficheros |
| d | Se han utilizado clases para almacenar información en ficheros |
| e | Se han utilizado clases para realizar conversiones entre diferentes formatos de ficheros |
| f | Se han previsto y gestionado las excepciones |

| CE | Criterio |
|---|---|
| g | Se han probado y documentado las aplicaciones desarrolladas |

## 3. Evaluación

- El RA1 pesa un **20% de la nota final**.
- Todos los criterios (a–g) se evalúan con **PEC (60%) + prueba objetiva (40%)**.
- **Prueba objetiva:** examen teórico-práctico; puede incluir preguntas tipo test, preguntas de desarrollo cortas o extensas y ejercicios prácticos en papel o en el ordenador.
- **PEC:** una o varias prácticas, individuales o por parejas según los requisitos. Hay que entregar una **memoria** y hacer una **defensa**; si no se saben responder las preguntas de la defensa, se considera **no apta**.
- **Materiales:** los apuntes y recursos están en el aula virtual. Hay una batería amplia de ejercicios que es **imprescindible realizar**, preguntando las dudas en clase.

# Mapa de contenidos (mapa mental)

El mapa mental organiza la unidad en tres partes. Sirve de esquema de lo que queda por ver en las siguientes tandas.

## Parte 1: JSON con Java

*Objetivo:* trabajar con JSON en Java usando distintas librerías (org.json, Gson y Jackson) para guardar y leer datos de ficheros o de aplicaciones.

| Bloque | Contenido |
|---|---|
| 1. Qué es JSON | Formato de intercambio, características, estructura (objetos y arrays), ventajas |
| 2. Librerías en Java | `org.json`, Gson (Google), Jackson (FasterXML); ¿cuál usar? |
| 3. Pasos principales | Dependencia Maven ( `pom.xml` ), crear estructura (objeto/array), guardar (serializar), leer (deserializar) |

| Bloque | Contenido |
|---|---|
| 4. Ejemplos de código | `DemoOrg.json.java`, `GuardarJSONv1.java`, `GuardarJSONv2.java`, `LeerJSONv1.java`, `LeerJSONv2.java` |
| 5. Casos prácticos | Objeto único, colección de objetos, ficheros JSON |
| 6. Comparativa | `org.json` vs Gson vs Jackson, ventajas e inconvenientes, cuándo usar cada una |

## Parte 2: JSON con Python

*Objetivo:* trabajar con JSON en Python con el módulo estándar `json`, para guardar y leer datos de ficheros o de streams.

| Bloque | Contenido |
|---|---|
| 1. Qué es JSON | Formato de intercambio, estructura, ventajas y usos |
| 2. Módulo `json` | Parte de la librería estándar, no necesita instalación, `import json` |
| 3. Operaciones principales | `json.dump()` / `json.dumps()` (guardar), `json.load()` / `json.loads()` (leer), trabajar con diccionarios y listas |
| 4. Ejemplos de código | Escribir un objeto JSON, leer un objeto JSON, escribir una lista de objetos, leer una lista de objetos |
| 5. Ficheros y streams | Guardar en fichero, leer desde fichero, leer desde una cadena de texto (string o stream) |
| 6. Java vs Python | Similitudes y diferencias, ventajas del enfoque de Python, cuándo usar cada uno |

## Parte 3: XML con Java

*Objetivo:* trabajar con XML en Java, leer y escribir ficheros XML, utilizar XPath y validar documentos.

| Bloque | Contenido |
|---|---|
| 1. Qué es XML | Lenguaje de marcado, características, ventajas y usos |
| 2. Estructura de un XML | Elementos y etiquetas, atributos, **bien formado vs válido**, DTD y XSD |
| 3. XPath | Qué es XPath, sintaxis básica, selección de nodos |
| 4. XML con Java | APIs principales (**DOM, SAX, StAX**), leer ficheros XML, escribir ficheros XML, ejemplos de código |
| 5. Validación | DTD, XSD (XML Schema), comprobar documentos |
| 6. Casos prácticos | Leer información con XPath, guardar documentos XML, validar con DTD/XSD |

# Tema 1.1: Ficheros secuenciales y aleatorios, directorios, flujos y excepciones

## 1. Clase `File`: gestión de ficheros y directorios

La clase `File` representa la **ruta** a un fichero o carpeta. Un objeto `File` **no es el fichero en sí**, sino una abstracción de su ruta. Permite comprobar si existe, crearlo, borrarlo o recorrer su contenido si es un directorio.

| Método | Qué hace |
|---|---|
| `exists()` | Indica si el fichero existe |
| `createNewFile()` | Crea un fichero vacío |
| `delete()` | Borra el fichero o carpeta |
| `mkdir()` | Crea un directorio |
| `listFiles()` | Devuelve la lista de ficheros de un directorio |
| `getAbsolutePath()` | Devuelve la ruta absoluta |

```
File fichero = new File("datos.txt");
if (fichero.exists()) {
    System.out.println("El fichero existe.");
} else {
    System.out.println("No existe el fichero.");
}
```

## 2. Formas de acceso a un fichero

| | Acceso secuencial | Acceso aleatorio |
|---|---|---|
| Cómo funciona | Se lee o escribe en orden, de principio a fin | Se salta a una posición concreta para leer o escribir desde allí |
| Limitación | No se puede saltar a una posición | — |
| Cuándo | Es la forma más común y sencilla | Modificar registros concretos sin recorrer todo el fichero; ficheros binarios grandes |
| Ejemplos | Leer un fichero de texto con nombres de alumnos; copiar una imagen byte a byte | Acceder a un registro concreto de un fichero binario |
| Clases en Java | `FileReader` / `FileWriter`, `BufferedReader` / `PrintWriter`, `FileInputStream` / `FileOutputStream` | `RandomAccessFile` |

### `RandomAccessFile`

- Constructor: `new RandomAccessFile(String nombreFichero, String modo)`.
- Modos: `"r"` (solo lectura) y `"rw"` (lectura y escritura).
- Lectura y escritura por tipos: `readInt()`, `readDouble()`, `readUTF()`... y `writeInt(int)`, `writeDouble(double)`, `writeUTF(String)`...
- Control del puntero:
  - `seek(long pos)`: mueve el puntero a la posición indicada (en bytes).
  - `getFilePointer()`: devuelve la posición actual.
  - `length()`: tamaño del fichero en bytes.

```
try (RandomAccessFile raf = new RandomAccessFile("datos.dat", "rw")) {
    raf.writeInt(101);        // ocupa 4 bytes
    raf.writeUTF("Pedro");    // cadena en formato UTF
    raf.writeDouble(8.5);     // ocupa 8 bytes

    raf.seek(0);              // volver al inicio del fichero

    int id = raf.readInt();
    String nombre = raf.readUTF();
    double nota = raf.readDouble();
    System.out.println("ID: " + id + ", Nombre: " + nombre + ", Nota: " +
nota);
} catch (IOException e) {
    e.printStackTrace();
}
```

Aquí no se recorre todo el fichero: se controla dónde leer y escribir gracias a `seek()`.

## 3. Flujos de bytes y flujos de caracteres

| | Flujos de bytes | Flujos de caracteres |
|---|---|---|
| Clase base | `InputStream` / `OutputStream` | `Reader` / `Writer` |
| Unidad de trabajo | 1 byte (8 bits) | 1 carácter (16 bits, Unicode) |
| Uso principal | Ficheros binarios (imágenes, audio, vídeo, PDF, ejecutables) | Ficheros de texto ( `.txt` , `.csv` , `.xml` , `.json` ) |
| Clases básicas | `FileInputStream` , `FileOutputStream` | `FileReader` , `FileWriter` |
| Con buffer | `BufferedInputStream` , `BufferedOutputStream` | `BufferedReader` , `BufferedWriter` |
| Avanzadas | `DataInputStream` / `DataOutputStream` (primitivos), `ByteArrayInputStream` / `ByteArrayOutputStream` (memoria), `ObjectInputStream` / `ObjectOutputStream` (objetos) | `PrintWriter` (escritura cómoda), `CharArrayReader` / `CharArrayWriter` (memoria), `StringReader` / `StringWriter` (cadenas en memoria) |
| Conversión | No convierte nada: bytes "en crudo" | Convierte automáticamente bytes ↔ caracteres Unicode |

| | Flujos de bytes | Flujos de caracteres |
|---|---|---|
| Ejemplo típico | Copiar una imagen ( `FileInputStream` → `FileOutputStream` ) | Leer un `.csv` con `BufferedReader` línea a línea |
| Cuándo usar | Datos binarios o no interpretables como texto | Contenido legible por humanos |

**Por qué no usar `Reader` / `Writer` con binarios:** convierten bytes ↔ caracteres Unicode y esa conversión rompería un PNG o un PDF. Con flujos de bytes lo que hay en disco es exactamente lo que se lee o escribe.

## 3.1 Flujos de bytes

**Métodos comunes:** `read()` (lee un byte, `-1` al final), `read(byte[] buffer)`, `write(int b)`, `write(byte[] buffer)`, `close()`.

### `FileInputStream` / `FileOutputStream`

Flujos de bytes conectados a un fichero. Sirven para leer o escribir binarios directamente desde o hacia disco (copiar imágenes, abrir PDF, mover ficheros binarios).

- Constructores: `new FileInputStream(File|String)`, `new FileOutputStream(File|String)` y `new FileOutputStream(..., true)` (**modo append**).
- Métodos clave: `int read()`, `int read(byte[] buf)`, `void write(int b)`, `void write(byte[] buf, int off, int len)`, `void close()`.

```
try (OutputStream out = new FileOutputStream("log.bin", true)) {   // append
    out.write(new byte[]{1, 2, 3, 4});
}
```

### `BufferedInputStream` / `BufferedOutputStream`

Decoradores que envuelven a un `InputStream` / `OutputStream` y añaden un **buffer en memoria**: menos accesos a disco y más rapidez. Conviene usarlos casi siempre con ficheros medianos o grandes.

- Constructores: `new BufferedInputStream(InputStream in, int tamBufferOpcional)` y `new BufferedOutputStream(OutputStream out, int tamBufferOpcional)`.
- Adicionales: `mark(int readLimit)` y `reset()` (si `markSupported()` es `true`); `flush()` en `BufferedOutputStream` vacía el buffer.

```
try (InputStream in = new BufferedInputStream(new
FileInputStream("video.mp4"))) {
    byte[] buf = new byte[16_384];          // 16 KiB
    while (in.read(buf) != -1) { /* procesar */ }
}
```

### `DataInputStream` / `DataOutputStream`

Flujos de bytes con métodos para **tipos primitivos y cadenas** en formato binario portable (no texto). Sirven para guardar o cargar `int`, `double`, `boolean` o UTF sin diseñar un formato propio (guardar registros binarios).

- Constructores: `new DataInputStream(InputStream in)` y `new DataOutputStream(OutputStream out)`.
- Lectura: `readInt()`, `readDouble()`, `readBoolean()`, `readUTF()`… Escritura: `writeInt(int)`, `writeDouble(double)`, `writeBoolean(boolean)`, `writeUTF(String)`…

```
// Escribir
try (DataOutputStream dos = new DataOutputStream(new
FileOutputStream("registro.bin"))) {
    dos.writeInt(42);
    dos.writeUTF("Ana");
    dos.writeDouble(9.1);
}
// Leer (en el mismo orden en que se escribió)
try (DataInputStream dis = new DataInputStream(new
FileInputStream("registro.bin"))) {
    int id = dis.readInt();
    String nombre = dis.readUTF();
    double nota = dis.readDouble();
}
```

### `ByteArrayInputStream` / `ByteArrayOutputStream`

Flujos de bytes **en memoria** (no tocan el disco). Útiles para transformar datos antes de escribirlos, para tests o cuando se necesita un buffer dinámico. En `ByteArrayOutputStream`, `byte[] toByteArray()` devuelve todo lo escrito.

```
ByteArrayOutputStream baos = new ByteArrayOutputStream();
baos.write(new byte[]{10, 20, 30});
byte[] datos = baos.toByteArray();
try (OutputStream out = new FileOutputStream("salida.bin")) {
    out.write(datos);
}
```

### `ObjectInputStream` / `ObjectOutputStream`

Flujos binarios orientados a objetos (**serialización Java**).

- **Serializar:** convertir un objeto en una secuencia de bytes para guardarlo en un fichero o transmitirlo por red.
- **Deserializar:** reconstruir el objeto original a partir de esos bytes.
- Permite persistir información de forma sencilla sin diseñar un formato propio.

```
Alumno alumno = new Alumno("Pedro", 22);
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("alumno.obj"))) {
    oos.writeObject(alumno);     // serializamos el objeto
    System.out.println("Objeto Alumno guardado en alumno.obj");
} catch (IOException e) {
    e.printStackTrace();
}
```

### Resumen de flujos de bytes

| Clase | Uso principal | Métodos comunes | Cuándo usar |
|---|---|---|---|
| `FileInputStream` | Leer binarios desde fichero | `read()`, `read(byte[])`, `close()` | Imágenes, audio, PDF, ejecutables |
| `FileOutputStream` | Escribir binarios en fichero | `write(int)`, `write(byte[])`, `close()` | Guardar o copiar binarios; admite append |
| `BufferedInputStream` | Lectura eficiente con buffer | `read()`, `read(byte[])`, `mark()`, `reset()`, `close()` | Envuelve un `InputStream` para menos accesos a disco |

| Clase | Uso principal | Métodos comunes | Cuándo usar |
|---|---|---|---|
| `BufferedOutputStream` | Escritura eficiente con buffer | `write(int)`, `write(byte[])`, `flush()`, `close()` | Acumula en memoria y escribe en bloque |
| `DataInputStream` | Leer primitivos en binario | `readInt()`, `readDouble()`, `readUTF()`, `close()` | Leer tipos nativos sin transformarlos a mano |
| `DataOutputStream` | Escribir primitivos en binario | `writeInt()`, `writeDouble()`, `writeUTF()`, `close()` | Guardar datos en binario portable |
| `ByteArrayInputStream` | Leer bytes desde un array en memoria | `read()`, `read(byte[])`, `available()`, `close()` | Pruebas o datos ya en memoria |
| `ByteArrayOutputStream` | Escribir bytes en un array dinámico | `write(int)`, `write(byte[])`, `toByteArray()`, `close()` | Construir un array antes de guardarlo o enviarlo por red |
| `ObjectInputStream` | Deserializar objetos | `readObject()`, `close()` | Reconstruir objetos guardados con `ObjectOutputStream` |
| `ObjectOutputStream` | Serializar objetos | `writeObject(Object o)`, `close()` | Guardar objetos completos en binario |

## 3.2 Flujos de caracteres

Trabajan con texto: convierten automáticamente los bytes del disco a caracteres Unicode. Son ideales para `.txt`, `.csv`, `.xml` y `.json`.

**Métodos comunes:** `read()` (un carácter, `-1` al final), `read(char[] buffer)`, `write(int c)`, `write(String s)`, `println(String s)` (en `PrintWriter`), `close()`.

### `FileReader`

Lee texto plano **carácter a carácter**. Convierte bytes a caracteres según la codificación del sistema (por defecto UTF-8 o la del SO).

```
try (FileReader fr = new FileReader("texto.txt")) {
    int c;
    while ((c = fr.read()) != -1) {
        System.out.print((char) c);      // convertimos int a char
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

### `FileWriter`

Complemento de `FileReader`: escribe texto carácter a carácter. Constructores: `new FileWriter("archivo.txt")` **sobrescribe** y `new FileWriter("archivo.txt", true)` **añade al final** (append). Métodos: `write(int c)`, `write(String s)`, `write(char[] buffer)`, `close()`.

```
try (FileWriter fw = new FileWriter("salida.txt")) {
    fw.write("Primera línea\n");
    fw.write("Segunda línea\n");
} catch (IOException e) {
    e.printStackTrace();
}
```

### `BufferedReader`

Decorador que envuelve un `Reader` y lo hace más eficiente (buffer en memoria, menos accesos a disco). Su método estrella es **`readLine()`**, que devuelve una línea completa sin el salto de línea (y `null` al terminar).

```
try (BufferedReader br = new BufferedReader(new FileReader("texto.txt"))) {
    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

### `PrintWriter`

Decorador que envuelve un `Writer` y ofrece `print()`, `println()` y `printf(String formato, Object... args)` (con formato, como en C). Muy usado en informes y logs.

```
try (PrintWriter pw = new PrintWriter(new FileWriter("salida.txt"))) {
    pw.println("Primera línea");
    pw.println("Segunda línea");
    pw.printf("La nota media es %.2f%n", 8.75);
} catch (IOException e) {
    e.printStackTrace();
}
```

### Resumen de flujos de caracteres

| Clase | Uso principal | Métodos comunes | Cuándo usar |
|---|---|---|---|
| `FileReader` | Leer texto carácter a carácter | `read()`, `read(char[])`, `close()` | Ficheros pequeños de texto; convierte bytes → caracteres |
| `FileWriter` | Escribir texto carácter a carácter | `write(int)`, `write(String)`, `write(char[])`, `close()` | Generar ficheros de texto; sobrescribir o añadir |
| `BufferedReader` | Lectura eficiente con buffer | `readLine()`, `read()`, `ready()`, `close()` | Leer texto línea a línea (`.txt`, `.csv`) |
| `BufferedWriter` (opcional) | Escritura eficiente con buffer | `write(String)`, `newLine()`, `flush()`, `close()` | Alternativa a `FileWriter` con eficiencia y control de líneas |
| `PrintWriter` | Escritura cómoda de texto | `print()`, `println()`, `printf()`, `close()` | Informes, logs o salidas con formato |

## 4. NIO.2 (`java.nio.file`)

Desde **Java 7** existe NIO.2, una API más moderna, flexible y potente que la clásica `java.io`. No la sustituye del todo, pero ofrece mayor rendimiento en algunas operaciones, mejor manejo de rutas y sistemas de ficheros, operaciones atómicas y seguras y soporte multiplataforma más uniforme.

### `Path`

Representa una ruta (absoluta o relativa) y sustituye a `File` en la mayoría de usos. Se obtiene con `Paths.get(...)`.

```
Path ruta = Paths.get("docs/ejemplo.txt");
System.out.println("Nombre: " + ruta.getFileName());
System.out.println("Directorio padre: " + ruta.getParent());
```

### `Files`

Clase de utilidades estáticas para operar con `Path`:

| Tarea | Métodos |
|---|---|
| Crear y borrar | `createFile()`, `createDirectory()`, `delete()` |
| Copiar y mover | `copy()`, `move()` |
| Comprobar | `exists()`, `isDirectory()`, `isRegularFile()` |
| Leer y escribir | `readAllLines()`, `write()`, `lines()` (stream) |

```
Path f = Paths.get("ejemplo.txt");
if (Files.notExists(f)) {
    Files.createFile(f);
}
Files.write(f, List.of("Hola", "Mundo"), StandardCharsets.UTF_8);
List<String> lineas = Files.readAllLines(f, StandardCharsets.UTF_8);
```

### `DirectoryStream`

Itera sobre el contenido de un directorio (similar a `listFiles()`), con la ventaja de que **no carga todos los elementos en memoria**.

```
Path dir = Paths.get("docs");
try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir)) {
    for (Path p : stream) {
        System.out.println(p.getFileName());
    }
}
```

### `FileVisitor` y `Files.walkFileTree()`

Para recorrer árboles de directorios **de forma recursiva**; se implementa una clase que extienda `SimpleFileVisitor<Path>`.

```
Files.walkFileTree(Paths.get("docs"), new SimpleFileVisitor<>() {
    @Override
    public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) {
        System.out.println("Fichero: " + file);
        return FileVisitResult.CONTINUE;
    }
});
```

### Comparativa `java.io` vs NIO.2

```
// java.io
File f = new File("ejemplo.txt");
if (!f.exists()) {
    f.createNewFile();
}

// NIO.2
Path f = Paths.get("ejemplo.txt");
if (Files.notExists(f)) {
    Files.createFile(f);
}
```

## 5. Manejo de excepciones

En las operaciones con ficheros pueden fallar muchas cosas: fichero inexistente, permisos insuficientes, disco lleno, rutas inválidas... En Java se gestionan con **excepciones comprobadas** (*checked*); la más habitual es `IOException`.

**Jerarquía básica:**

- `IOException`: excepción general de entrada/salida.
  - `FileNotFoundException`: fichero no encontrado.
  - `EOFException`: fin de fichero inesperado.
  - `UnsupportedEncodingException`: codificación de caracteres no soportada.

## Formas de gestionarlas

**1. `try` – `catch` clásico.** Captura la excepción y permite tratarla sin interrumpir el programa. Se pueden poner varios `catch` para distintos tipos; `finally` es opcional y **se ejecuta siempre** (ideal para liberar recursos).

```
FileReader fr = null;
try {
    fr = new FileReader("ejemplo.txt");
    int caracter;
    while ((caracter = fr.read()) != -1) {
        System.out.print((char) caracter);
    }
} catch (IOException e) {
    System.err.println("Error de E/S: " + e.getMessage());
} finally {
    if (fr != null) {                       // cierre manual para evitar
fugas
        try {
            fr.close();
        } catch (IOException e) {
            System.err.println("Error al cerrar el fichero: " +
e.getMessage());
        }
    }
}
```

**2. `try-with-resources` (recomendado desde Java 7).** Garantiza que el recurso se cierre automáticamente. Todo recurso declarado entre paréntesis debe implementar `AutoCloseable` (`FileReader`, `FileWriter`, `BufferedReader`, `Scanner`, `Connection`…). Se pueden declarar varios separados por `;`.

```
try (FileReader fr = new FileReader("ejemplo.txt")) {
    int caracter;
    while ((caracter = fr.read()) != -1) {
        System.out.print((char) caracter);
    }
    System.out.println("\nLectura finalizada.");
} catch (IOException e) {
    System.err.println("Error de E/S: " + e.getMessage());
}
```

Diferencias con el clásico: el recurso se declara en la cabecera del `try` (no hay `= null` fuera), no hace falta `finally` para cerrar y el código es más corto y menos propenso a errores.

**3. Propagar la excepción** con `throws`, cuando no se quiere gestionar en ese método:

```
public static void leerFichero(String nombre) throws IOException {
    try (FileReader fr = new FileReader(nombre)) {
        // ...
    }
}
```

### Buenas prácticas

- Usar siempre **try-with-resources** para evitar fugas (no olvidar cerrar el fichero).
- Dar **mensajes de error claros** al usuario (por ejemplo, "No se encuentra el fichero alumnos.txt").
- Evitar capturar `Exception` de forma genérica salvo que sea estrictamente necesario.
- **Registrar (log)** los errores en aplicaciones grandes.

## 6. Resumen para repasar

| Caso | Clave |
|---|---|
| Representar una ruta (clásico) | `File` (abstracción de la ruta, no el fichero) |
| Representar una ruta (moderno) | `Path` + `Files` (NIO.2, desde Java 7) |
| Acceso secuencial | Flujos (`Reader`/`Writer`, `InputStream`/`OutputStream`), de principio a fin |
| Acceso aleatorio | `RandomAccessFile` con `seek()`; modos `"r"` y `"rw"` |
| Binarios (imágenes, PDF...) | Flujos de **bytes**, nunca `Reader`/`Writer` |
| Texto (`.txt`, `.csv`, `.xml`, `.json`) | Flujos de **caracteres** |
| Leer texto línea a línea | `BufferedReader.readLine()` (devuelve `null` al final) |
| Escribir texto con formato | `PrintWriter` (`println`, `printf`) |
| Añadir al final | `new FileWriter(f, true)` / `new FileOutputStream(f, true)` |
| Tipos primitivos en binario | `DataInputStream` / `DataOutputStream` (leer en el mismo orden que se escribió) |

| Caso | Clave |
|---|---|
| Objetos | `ObjectOutputStream.writeObject()` / `ObjectInputStream.readObject()` |
| Rendimiento | Decoradores `Buffered...` |
| Recorrer directorios | `listFiles()` (clásico), `DirectoryStream` o `walkFileTree()` (NIO.2) |
| Excepciones | `IOException` (checked); try-with-resources; `throws` para propagar |

## 7. Cosas a tener en cuenta

- La guía didáctica cita como contenidos solo JSON y XML, pero el mapa mental incluye también una parte de **JSON con Python**; conviene confirmar en clase si entra en la evaluación.
- Los archivos llegan numerados ( `1_1` , `1_3` ...): **faltaba el `1_2`** , que trataba la serialización y deserialización de objetos (contenido 2 de la guía). Ya está integrado como tema 1.2, más abajo en este documento.
- El PDF del tema 1.1 termina en mitad de una frase en las buenas prácticas ("...aplicaciones grande"), probablemente por un fallo de maquetación; se ha completado como "aplicaciones grandes".
- Los ejemplos de serialización usan una clase `Alumno` que no se muestra en el PDF; para serializarla debe implementar `Serializable` .

# Tema 1.2: Serialización y deserialización de objetos

## 1. Qué es serializar y deserializar

- **Serializar** = convertir un objeto en bytes para guardarlo (fichero, BBDD, caché) o enviarlo por red.
- **Deserializar** = reconstruir el objeto desde esos bytes.
- **Se usa para:** persistencia rápida de objetos (formato binario), comunicación entre procesos y cachés o colas (cuando no se usan JSON, XML o protocolos).

**En Java:**

- `ObjectOutputStream` → escribe objetos con `writeObject` .

- `ObjectInputStream` → lee objetos con `readObject`.
- La clase debe implementar `java.io.Serializable` (interfaz **marcador**, sin métodos).

**Términos clave:**

- Todos los atributos deben ser serializables.
- Si un atributo no lo es (p. ej., un `Socket`), se marca como **`transient`**.
- `transient` = no se guarda en la secuencia de bytes; al deserializar queda con su valor por defecto (`null`, `0`, `false`…).

## 2. Serializar un objeto a fichero

La clase `Persona` debe implementar `Serializable`:

```
package model;
import java.io.Serializable;

public class Persona implements Serializable {
    // Recomendado para el control de versión de la clase (ver apartado 6)
    private static final long serialVersionUID = 1L;

    private String nombre;
    private int edad;
    private transient String password;   // No se serializa

    public Persona(String nombre, int edad, String password) {
        this.nombre = nombre;
        this.edad = edad;
        this.password = password;
    }

    @Override
    public String toString() {
        return nombre + " (" + edad + " años) pwd=" + password;
    }
}
```

```
public class DemoSerializarUno {
    public static void main(String[] args) {
        Persona p = new Persona("Ana", 25, "secreto123");
        try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream("persona.dat"))) {
            oos.writeObject(p);                      // Serializa 'p'
            System.out.println("OK: Objeto escrito en persona.dat");
        } catch (IOException e) {
            System.err.println("Error escribiendo: " + e.getMessage());
        }
    }
}
```

`FileOutputStream` abre el fichero para escribir bytes; `ObjectOutputStream` lo "envuelve" para convertir objetos en bytes. `writeObject(obj)` hace la conversión interna y la envía al fichero.

**Pasos:** (1) marcar la clase con `Serializable`; (2) abrir un `ObjectOutputStream`; (3) llamar a `writeObject()`; (4) cerrar el flujo para no perder datos.

## 3. Deserializar un objeto desde fichero

Paso inverso: hace falta un fichero (p. ej. `persona.dat`) generado antes con `writeObject()`.

```
public class DemoDeserializarUno {
    public static void main(String[] args) {
        try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream("persona.dat"))) {
            Persona p = (Persona) ois.readObject();
            System.out.println("Leído: " + p);       // password será null
(transient)
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("Error leyendo: " + e.getMessage());
        }
    }
}
```

`FileInputStream` abre el fichero para leer bytes y `ObjectInputStream` los convierte en objetos. `readObject()` devuelve `Object`, así que hay que hacer **cast** al tipo original.

**Pasos:** (1) abrir `ObjectInputStream`; (2) `readObject()`; (3) cast al tipo correcto; (4) cerrar el flujo; (5) usar el objeto como si se acabara de crear en memoria.

# 4. Serializar y deserializar varios objetos

Hay dos opciones: **A** (objetos sueltos uno detrás de otro, leyendo hasta `EOFException`) y **B** (una colección entera, p. ej. `ArrayList<Persona>`).

| Aspecto | Opción A: objetos sueltos (EOF) | Opción B: colección entera |
|---|---|---|
| Fichero del ejemplo | `personas.dat` | `personas_lista.dat` |
| Escritura | Bucle con `writeObject(p)` por cada objeto | Un solo `writeObject(lista)` |
| Lectura | `while(true)` con `readObject()`; termina capturando `EOFException` | Un único `readObject()`, cast a `List<Persona>` y recorrer |
| Control de flujo | Depende de una excepción para saber cuándo parar | No depende de excepciones |
| Claridad del código | Menos limpia | Más clara y mantenible |
| Añadir objetos | Fácil con append (sin cabecera extra, ver apartado 5) | Hay que reescribir toda la colección |
| Grandes volúmenes | Mejor: se procesa uno a uno (streaming) | Malo: se carga todo en memoria |
| Acceso a un objeto concreto | Difícil sin recorrer todos | Lista completa lista para usar |
| Casos de uso | Logs, streaming, escritura incremental | Persistencia de listas completas |

```
// OPCIÓN A: escribir seguidos y leer hasta EOF
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("personas.dat"))) {
    for (Persona p : personas) {
        oos.writeObject(p);
    }
}
try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("personas.dat"))) {
    while (true) {
        Persona p = (Persona) ois.readObject();
        System.out.println("Leída: " + p);
    }
} catch (EOFException eof) {
    System.out.println("Fin de fichero.");
}
```

```
// OPCIÓN B: escribir y leer la colección entera
try (ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream("personas_lista.dat"))) {
    oos.writeObject(personas);                // una sola llamada
}
try (ObjectInputStream ois = new ObjectInputStream(
        new FileInputStream("personas_lista.dat"))) {
    List<Persona> leidas = (List<Persona>) ois.readObject();
    for (Persona p : leidas) {
        System.out.println(" - " + p);
    }
}
```

**Conclusión:** para persistir un conjunto completo → colección (B); para guardar objetos de forma incremental, como un log → objetos seguidos con EOF (A).

## 5. Añadir objetos a un fichero existente (sin reescribir la cabecera)

Al crear un `ObjectOutputStream`, su constructor escribe automáticamente al inicio del flujo una **cabecera** con la firma y versión del formato de serialización:

- `STREAM_MAGIC` = `0xAC ED`
- `STREAM_VERSION` = `0x00 05`

La cabecera permite a `ObjectInputStream` verificar que es un flujo de objetos Java válido. **El problema:** si se abre el fichero en modo append y se crea otro `ObjectOutputStream`, este vuelve a escribir la cabecera en mitad del fichero. Al leer,

`ObjectInputStream` espera un objeto y encuentra otra cabecera ( `AC ED 00 05` ), y lanza `StreamCorruptedException: invalid stream header` (o "unexpected block data").

### Patrón A: mantener un único `ObjectOutputStream` abierto

Se abre una vez, se llama a `writeObject()` las veces necesarias y se cierra al final. No se escriben cabeceras adicionales porque no se recrea el flujo. Útil en procesos "vivos" (logs, colas) o al exportar una lista de golpe, pero **no sirve** si mañana hay que reabrir el programa para añadir más (haría falta un nuevo flujo y se reescribiría la cabecera). Aquí no hay append real: todo se escribe en una sola ejecución.

```
try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("patronA.dat"))) {
    for (Persona p : personas) {
        oos.writeObject(p);             // cada llamada serializa un objeto
    }
}   // al salir del try-with-resources se cierra automáticamente
```

### Patrón B: reabrir en append sin reescribir la cabecera

Primera escritura con un `ObjectOutputStream` normal (deja la cabecera). Las siguientes, con un flujo especial que **omite la cabecera**, creado al subclasear `ObjectOutputStream` y sobrescribir `writeStreamHeader()` :

```
public class AppendingObjectOutputStream extends ObjectOutputStream {
    public AppendingObjectOutputStream(OutputStream out) throws IOException {
        super(out);
    }
    @Override
    protected void writeStreamHeader() throws IOException {
        reset();   // evita escribir STREAM_MAGIC/STREAM_VERSION otra vez
    }
}
```

```
public class AnadirObjetosAppendSinCabecera {
    private static final String FICHERO = "patronB.dat";

    public static void main(String[] args) {
        escribir(List.of(new Persona("Ana", 25, "a"), new Persona("Luis", 31,
"b")));  // 1ª tanda
        escribir(List.of(new Persona("Marta", 28, "c")));    // 2ª tanda: sin
reescribir cabecera
    }
```

```
    private static void escribir(List<Persona> personas) {
        File f = new File(FICHERO);
        boolean existe = f.exists() && f.length() > 0;
        try (OutputStream fos = new FileOutputStream(f, true);
             ObjectOutputStream oos = existe
                     ? new AppendingObjectOutputStream(fos)   // sin cabecera
                     : new ObjectOutputStream(fos)) {         // con cabecera
            for (Persona p : personas) {
                oos.writeObject(p);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Al leer con `ObjectInputStream` todo se interpreta bien, porque solo hay una cabecera al inicio. Sirve, por ejemplo, para un programa que se ejecuta cada día y añade registros.

| Patrón A | Patrón B |
|---|---|
| Se abre un solo `ObjectOutputStream` | Se puede reabrir el fichero en sesiones distintas |
| No hay riesgo de cabecera extra | Se usa un `ObjectOutputStream` especial sin cabecera |
| Solo sirve si se escriben todos los objetos en una ejecución | Sirve para añadir objetos en varias ejecuciones |

## 6. `serialVersionUID`

Al serializar, Java guarda la **versión de la clase** junto a los datos: un número llamado `serialVersionUID`, el "DNI" de la clase. Al deserializar, Java comprueba que la clase del código coincide con la usada al escribir el fichero.

- **Si no se declara,** Java lo genera automáticamente a partir de la estructura de la clase. Si luego se cambia la clase (añadir o renombrar un atributo, cambiar modificadores...), el UID calculado cambia, y al leer un fichero antiguo se lanza `java.io.InvalidClassException: model.Persona; local class incompatible`.
- **Si se declara,** se controla la compatibilidad: aunque se añada un campo, mientras se mantenga el mismo `serialVersionUID`, Java permite leer los objetos guardados con la versión anterior, y los campos nuevos se inicializan a su valor por defecto (`null`, `0`, `false`).

```
private static final long serialVersionUID = 1L;
```

## 7. Buenas prácticas

- Usar siempre **try-with-resources**: cierra los flujos automáticamente.
- Usar colecciones enteras (opción B) cuando los objetos se gestionan en bloque.
- Opción A (EOF) solo cuando haga falta un "log" incremental.
- Evitar serializar objetos muy grandes o dependientes de red (p. ej. `Socket`). Para interoperabilidad, mejor JSON, XML, etc.

## 8. Resumen para repasar

| Concepto | Clave |
|---|---|
| Serializar / deserializar | Objeto → bytes / bytes → objeto |
| Clases | `ObjectOutputStream.writeObject()` / `ObjectInputStream.readObject()` (con cast) |
| Requisito | `implements Serializable` (interfaz marcador) y atributos serializables |
| `transient` | No se guarda; vuelve a `null` / `0` / `false` |
| Varios objetos | A: sueltos hasta `EOFException` (log). B: colección entera (persistencia en bloque) |
| Cabecera `AC ED 00 05` | La escribe cada `ObjectOutputStream` nuevo; en append provoca `StreamCorruptedException` |
| Añadir sin cabecera | Patrón A (un solo flujo) o patrón B (`writeStreamHeader()` sobrescrito) |
| `serialVersionUID` | Fijarlo para controlar la compatibilidad; sin él, cualquier cambio rompe la lectura (`InvalidClassException`) |

## 9. Cosas a tener en cuenta

- El PDF remite a "ver 6.1" para `serialVersionUID`, pero ese apartado es el 2.6 del propio PDF (cada archivo numera sus apartados por separado).

- Mantener el mismo `serialVersionUID` solo permite leer ficheros antiguos tras cambios **compatibles** (como añadir campos); otros cambios, como modificar el tipo de un atributo, siguen rompiendo la lectura.
- Los ejemplos de la opción B hacen un cast a `List<Persona>` sin comprobar el tipo, por lo que el compilador avisa con un *unchecked warning*.

# Tema 1.3: Ficheros de intercambio de datos (JSON y XML)

## 1. Introducción

Los ficheros binarios y la serialización de Java usan un formato propietario: no es legible fácilmente por humanos ni interoperable con otros lenguajes. Los **formatos de intercambio de datos** estandarizados cumplen tres características:

- **Legibles:** se abren en un editor de texto y se entienden.
- **Independientes del lenguaje** de programación.
- **Multiplataforma:** permiten intercambiar información entre aplicaciones de escritorio, servidores web y dispositivos móviles.

| Formato | Características |
|---|---|
| **JSON** (JavaScript Object Notation) | Sintaxis ligera basada en pares clave-valor y arrays; muy usado en APIs REST, microservicios y aplicaciones web; compacto y rápido de procesar |
| **XML** (eXtensible Markup Language) | Lenguaje de marcado jerárquico con etiquetas; más verboso que JSON, pero muy extendido en sistemas empresariales y configuraciones; permite validación con DTD y XSD |

Ambos sustituyen, en la mayoría de los casos, a la serialización binaria, porque permiten compartir datos entre aplicaciones en lenguajes distintos, guardar configuraciones y datos de forma comprensible, y servir de soporte a la comunicación cliente-servidor.

Un objeto `Alumno` con nombre y edad: en binario (serializado en Java) serían bytes ilegibles; en JSON y en XML:

```
{ "nombre": "Ana", "edad": 20 }
```

```
<alumno>
   <nombre>Ana</nombre>
   <edad>20</edad>
</alumno>
```

## 2. JSON

**JSON** es un formato de intercambio de datos basado en texto. Nació en el mundo web, pero se usa en cualquier aplicación que intercambie información entre sistemas. Es ligero, fácil de leer para las personas y fácil de procesar por las máquinas. Es el formato más habitual en APIs REST y servicios web.

- Representa los datos como pares **clave–valor**, con una sintaxis heredada de JavaScript pero independiente de él.
- Tipos básicos: cadenas ( `"Ana"` ), números ( `25` ), booleanos ( `true` , `false` ), nulos ( `null` ), arrays ( `[1,2,3]` ) y objetos ( `{"nombre":"Ana", "edad":25}` ).

```
{
    "nombre": "Ana",
    "edad": 25,
    "notas": [8.5, 7.2, 9.1],
    "activo": true
}
```

## 3. JSON en Java

Java **no trae soporte nativo completo** para JSON en la librería estándar, así que se usan librerías externas: `org.json` , Gson (Google) y Jackson (FasterXML).

### `org.json`

Librería básica, muy usada en proyectos pequeños y medianos. Convierte directamente entre objetos Java y JSON con una sintaxis clara. Clases principales: `JSONObject` , `JSONArray` y `JSONTokener` .

**`JSONObject`** : representa un objeto JSON (pares clave–valor).

- `put(String clave, Object valor)` : añade un par clave–valor.
- `getString("clave")` , `getInt("clave")` , `getBoolean("clave")` : obtienen el valor asociado.
- `optString("clave", valorPorDefecto)` : como `getString` , pero **no lanza excepción** si la clave no existe.

- `toString()`: devuelve el contenido como texto JSON.

```
JSONObject persona = new JSONObject();
persona.put("nombre", "Ana");
persona.put("edad", 25);
System.out.println(persona.toString());   // {"nombre":"Ana","edad":25}
```

**`JSONArray`**: representa un array JSON (lista de objetos o valores).

- `put(Object valor)`: añade un elemento.
- `getJSONObject(int index)`: devuelve el objeto de esa posición.
- `length()`: número de elementos. `isEmpty()`: indica si está vacío.

```
JSONArray lista = new JSONArray();
JSONObject p1 = new JSONObject().put("nombre", "Ana").put("edad", 25);
JSONObject p2 = new JSONObject().put("nombre", "Luis").put("edad", 30);
lista.put(p1);
lista.put(p2);
System.out.println(lista.toString(4));    // con sangría de 4 espacios
```

**`JSONTokener`**: interpreta un flujo de texto o fichero y lo convierte en tokens JSON; útil al leer JSON desde ficheros o streams. Método clave: `nextValue()`, que devuelve el primer valor JSON válido (`JSONObject`, `JSONArray` o valor simple).

```
try (BufferedReader br = Files.newBufferedReader(Path.of("personas.json"))) {
    JSONTokener tokener = new JSONTokener(br);
    Object raiz = tokener.nextValue();
    if (raiz instanceof JSONArray) {
        JSONArray array = (JSONArray) raiz;
        System.out.println("Total personas: " + array.length());
    }
}
```

**Dependencia Maven:**

```
<dependency>
    <groupId>org.json</groupId>
    <artifactId>json</artifactId>
    <version>20240303</version>
</dependency>
```

Si no se usa Maven, hay que descargar el `.jar` y agregarlo a las librerías del espacio de trabajo.

**Ejemplos del aula virtual (code_examples de JSON):**

| Fichero | Qué hace |
|---|---|
| `DemoOrgJson.java` | Crea un objeto `JSONObject` y lo muestra por consola |
| `GuardarJSONv1.java` | Crea un `JSONObject` y lo escribe en un fichero dentro del directorio `data` del proyecto |
| `LeerJSONv1.java` | Lee un fichero JSON con `JSONTokener` |
| `GuardarJSONv2.java` | Crea un `JSONArray` de objetos y lo guarda en un fichero dentro de `data` |
| `LeerJSONv2.java` | Lee un fichero JSON (array) con `JSONTokener` |

### Gson (Google)

Librería ligera y sencilla, muy usada en proyectos pequeños y medianos. Su ventaja principal: **convierte automáticamente** entre objetos Java y JSON sin construir a mano `JSONObject` o `JSONArray`; basta con indicar la clase del objeto.

- Clase principal: `Gson`.
- `toJson(objeto)`: objeto Java → cadena JSON.
- `fromJson(cadenaJson, Clase.class)`: JSON → objeto Java de esa clase.

```
<dependency>
    <groupId>com.google.code.gson</groupId>
    <artifactId>gson</artifactId>
    <version>2.11.0</version>
</dependency>
```

Ejemplos del aula virtual: `DemoGson.java` (crea un `Alumno` y lo guarda en JSON), `GuardarAlumnosGson.java` (guarda una lista de `Alumno` en un fichero JSON) y `LeerAlumnosGson.java` (lee un fichero JSON y lo muestra por pantalla).

### Jackson (FasterXML)

Librería muy extendida en proyectos grandes y aplicaciones empresariales. Muy completa: soporta JSON, XML, YAML... Se integra con frameworks como **Spring Boot** (el temario lo marca como MUY IMPORTANTE). Su ventaja: gran potencia y más opciones de configuración que Gson.

```
<dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.2</version>
</dependency>
```

| Clase | Para qué |
|---|---|
| `ObjectMapper` | Clase principal: convierte JSON ↔ objetos Java. `writeValue(fichero, objeto)` guarda un objeto como JSON; `readValue(fichero, Clase.class)` lo lee |
| `ObjectWriter` | Variante para configurar la salida (p. ej. `writerWithDefaultPrettyPrinter()` para JSON legible) |
| `TypeReference<T>` | Necesario para deserializar colecciones genéricas (listas, mapas…) |

## ¿Cuándo usar cada librería?

| Librería | Clases principales | Características | Cuándo usarla |
|---|---|---|---|
| `org.json` | `JSONObject`, `JSONArray`, `JSONTokener` | Muy simple y ligera; sin configuración complicada; útil para crear y recorrer estructuras JSON básicas; no ofrece automatismos ni opciones avanzadas | Introducción a JSON, ejemplos didácticos, pruebas rápidas, proyectos muy pequeños |
| Gson (Google) | `Gson`, `GsonBuilder` | Convierte automáticamente objetos Java ↔ JSON; sintaxis clara y compacta; fácil con colecciones | Proyectos pequeños y medianos donde se busca rapidez y sencillez |

| Librería | Clases principales | Características | Cuándo usarla |
|---|---|---|---|
| Jackson (FasterXML) | `ObjectMapper`, `ObjectWriter`, `TypeReference` | La más potente y flexible; soporta JSON, XML, YAML; se integra con Spring Boot; mejor rendimiento con gran volumen de datos | Proyectos grandes, aplicaciones empresariales, APIs REST |

## Anexo: adaptadores personalizados en Gson

Gson convierte automáticamente los tipos comunes (`String`, `int`, `boolean`, listas...), pero **no reconoce** tipos complejos como `LocalDate` o `LocalDateTime`: `new Gson().toJson(fecha)` fallaría o daría algo extraño. Para esos casos se crea un **adaptador**: una clase que enseña a Gson a serializar (objeto Java → JSON) y deserializar (JSON → objeto Java).

| Interfaz | Qué hace |
|---|---|
| `JsonSerializer<T>` | Define cómo convertir un objeto Java `T` en JSON |
| `JsonDeserializer<T>` | Define cómo leer un JSON y convertirlo a un objeto `T` |

```
public class GsonLocalDateAdapter
        implements JsonSerializer<LocalDate>, JsonDeserializer<LocalDate> {

    private static final DateTimeFormatter FMT =
DateTimeFormatter.ISO_LOCAL_DATE;

    @Override
    public JsonElement serialize(LocalDate src, Type typeOfSrc,
JsonSerializationContext ctx) {
        return new JsonPrimitive(FMT.format(src));            // "2025-10-07"
    }

    @Override
    public LocalDate deserialize(JsonElement json, Type typeOfT,
                                 JsonDeserializationContext ctx) throws
JsonParseException {
        return LocalDate.parse(json.getAsString(), FMT);
    }
}
```

Y se registra el adaptador:

```
Gson gson = new GsonBuilder()
        .setPrettyPrinting()
        .registerTypeAdapter(LocalDate.class, new GsonLocalDateAdapter())
        .create();
```

Ejemplo del aula virtual `gson-adapters-demo` (proyecto Maven completo): `LocalDate` → `"yyyy-MM-dd"`, `LocalDateTime` → `"yyyy-MM-dd'T'HH:mm:ss"` y `Path` → `String` con la ruta del sistema.

### Anexo: cómo maneja Jackson los mismos tipos

Jackson sí reconoce los tipos de fecha y hora, **registrando el módulo adecuado**:

```
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule());                        //
LocalDate, LocalDateTime...
mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
mapper.enable(SerializationFeature.INDENT_OUTPUT);                  // pretty
printing
```

| Módulo Jackson | Qué añade |
|---|---|
| `jackson-datatype-jsr310` | Fechas y horas de Java 8 (`LocalDate`, `Instant`…) |
| `jackson-datatype-jdk8` | Tipos `Optional` y clases de Java 8 |
| `jackson-module-parameter-names` | Reconoce parámetros de constructores |
| `jackson-datatype-hibernate` | Entidades JPA/Hibernate |
| `jackson-datatype-json-org` | Compatibilidad con `org.json` |

## 4. JSON con Python

En Python el manejo de JSON es más sencillo que en Java: el módulo **`json`** viene en la librería estándar, y se trabaja directamente con cadenas, diccionarios y listas, sin instalar nada.

| Función | Qué hace |
|---|---|
| `json.dumps(obj)` | Convierte un objeto Python (diccionario, lista…) en una **cadena** JSON |

| Función | Qué hace |
|---|---|
| `json.dump(obj, fichero)` | Igual, pero escribe directamente en un **fichero** |
| `json.loads(cadena)` | Convierte una cadena JSON en un objeto Python (diccionario o lista) |
| `json.load(fichero)` | Igual, pero lee directamente desde un **fichero** JSON |
| `indent=4` (parámetro) | Genera JSON con formato legible (pretty printing) |

Regla para recordarlas: la **s** de `dumps` / `loads` es de *string*; sin la **s**, se trabaja con un fichero.

Ejemplos del aula virtual (json-Python): `01_basico_memoria.py`, `02_guardar_alumnos.py`, `03_leer_alumnos.py`, y `pathlib_example.py` (uso de `pathlib` que aparece en los anteriores).

## 5. JSON: Java vs Python

| Aspecto | Java | Python |
|---|---|---|
| Soporte nativo | No (librerías externas) | Sí ( `json` estándar) |
| Librerías típicas | `org.json` (básica), Gson (sencilla), Jackson (potente; estándar en Spring) | `json` estándar; opcionales: `pydantic`, `orjson` |
| Objeto → JSON (memoria) | `new Gson().toJson(obj)` / `new ObjectMapper().writeValueAsString(obj)` | `json.dumps(obj, indent=4)` |
| JSON → objeto (memoria) | `new Gson().fromJson(str, Clase.class)` / `new ObjectMapper().readValue(str, Clase.class)` | `json.loads(str)` |
| Fichero: escribir | `gson.toJson(obj, writer)` / `mapper.writeValue(file, obj)` | `json.dump(obj, file, indent=4)` |
| Fichero: leer | `gson.fromJson(reader, Tipo)` / `mapper.readValue(file, Clase.class)` | `json.load(file)` |
| Colecciones genéricas | Gson: `TypeToken<List<Alumno>>(){}`; Jackson: `new TypeReference<List<Alumno>>() {}` | Listas y diccionarios nativos; tipado opcional con `typing` / `dataclasses` |

| Aspecto | Java | Python |
|---|---|---|
| Modelos de datos | Clases con campos, getters/setters y (recomendable) constructor vacío | Diccionarios y listas; opcional `@dataclass` |
| Pretty printing | Gson: `GsonBuilder().setPrettyPrinting()`; Jackson: `writerWithDefaultPrettyPrinter()` | `indent=4` |
| Tipos no JSON (fecha/decimal) | Jackson: serializers/deserializers; Gson: `TypeAdapter` personalizados | `default=` en `dump(s)` y `object_hook` en `load(s)` |
| Validación de datos | Jackson + anotaciones / Bean Validation | `pydantic` o validaciones manuales |
| Streaming / grandes volúmenes | Jackson Streaming API (`JsonParser` / `JsonGenerator`) | `ijson`, JSON Lines (NDJSON) |
| Integración con frameworks | Excelente (Spring Boot usa Jackson por defecto) | Muy simple en scripts; frameworks como FastAPI/Flask |
| Rendimiento | Jackson suele ser más rápido que Gson | `json` suficiente; `orjson` / `ujson` para máximo rendimiento |
| Curva de aprendizaje | Mayor (mapeo a clases, genéricos) | Menor (estructuras nativas) |
| Cuándo elegir | Backends empresariales, APIs REST con Spring, contratos estrictos | Scripts, ETL ligeros, prototipos, data science; APIs rápidas con FastAPI |

# 6. XML

**XML** (Extensible Markup Language) es un lenguaje de marcado jerárquico con etiquetas y atributos: legible, multiplataforma y validable (DTD/XSD). Muy usado en configuración y sistemas empresariales. Frente a JSON es más verboso, pero ofrece **validación formal** y **namespaces** (espacios de nombres).

```
<?xml version="1.0" encoding="UTF-8"?>
<alumnos xmlns="http://ejemplo.org/alumnos">
  <alumno id="A001">
    <nombre>Ana</nombre>
    <edad>20</edad>
  </alumno>
  <alumno id="A002">
    <nombre>Luis</nombre>
    <edad>22</edad>
  </alumno>
</alumnos>
```

## Bien formado vs válido

| | Bien formado | Válido |
|---|---|---|
| Qué comprueba | Reglas **sintácticas**: la "gramática" del documento | Además de estar bien formado, cumple un **contrato (esquema)** |
| Reglas | Raíz única; etiquetas que se abren y cierran bien y bien anidadas; atributos entre comillas... | Qué elementos pueden aparecer y en qué orden; qué atributos son obligatorios u opcionales; cuántas veces se repiten; incluso tipos (número, fecha, patrón...) |
| Contenido y tipos | No los comprueba | Sí, según el esquema |

El contrato puede escribirse en **DTD** (Document Type Definition: más antiguo y menos expresivo) o **XSD** (XML Schema Definition: más moderno y tipado; soporta espacios de nombres, restricciones por patrón, rangos...).

**Ejemplo de DTD:**

```
<!-- La raíz es <alumnos> y debe contener uno o más <alumno> (el + significa
1..n) -->
<!ELEMENT alumnos (alumno+)>
<!-- Cada <alumno> contiene EXACTAMENTE, en ese orden, <nombre> y <edad> -->
<!ELEMENT alumno (nombre, edad)>
<!-- <alumno> tiene un atributo 'id' de tipo texto (CDATA), OBLIGATORIO
(#REQUIRED) -->
<!ATTLIST alumno id CDATA #REQUIRED>
<!-- #PCDATA = Parsed Character Data (solo texto) -->
<!ELEMENT nombre (#PCDATA)>
<!ELEMENT edad (#PCDATA)>
```

La DTD no tipa números; para tipos hay que usar XSD. XML válido para esa DTD:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE alumnos SYSTEM "alumnos.dtd">
<alumnos>
  <alumno id="A001">
    <nombre>Ana</nombre>
    <edad>20</edad>
  </alumno>
</alumnos>
```

Si se pone un `<alumno>` sin `<edad>`, el documento deja de ser válido (la DTD exige nombre y edad).

# 7. XPath

**XPath** (XML Path Language) es un lenguaje de consultas para **navegar y seleccionar partes de un documento XML**. Son "rutas" que apuntan a nodos (elementos, atributos, texto) dentro del árbol XML, igual que una ruta de carpetas apunta a un archivo.

**Para qué sirve:** localizar elementos y atributos concretos (p. ej., todos los `<alumno>` con edad > 20), filtrar por condiciones (predicados), posiciones o valores, obtener textos y contar nodos. Se usa junto con DOM/SAX o librerías de binding.

**Conceptos clave:**

- **Ruta:** el camino hasta los nodos ( `/` , `//` , nombres de etiqueta).
- **Predicados:** filtros entre corchetes `[ ... ]` para condiciones o posiciones.
- **Atributos:** se referencian con `@` (p. ej. `@id` ).
- **Funciones:** `text()` , `count()` , `contains()` ...
- **Contexto:** las rutas se evalúan desde un nodo de partida (normalmente la raíz).

Sobre este XML:

```
<alumnos>
  <alumno id="A001"><nombre>Ana</nombre><edad>20</edad></alumno>
  <alumno id="A002"><nombre>Luis</nombre><edad>22</edad></alumno>
</alumnos>
```

| Expresión XPath | Qué devuelve |
|---|---|
| `//alumno` | Todos los elementos `<alumno>` |
| `//alumno[@id="A002"]` | El `<alumno>` cuyo atributo `id` es "A002" |

| Expresión XPath | Qué devuelve |
|---|---|
| `//alumno[edad > 20]/nombre/text()` | El texto de `<nombre>` de los alumnos con `<edad>` > 20 |
| `count(//alumno)` | Número de alumnos |
| `//alumno[1]` | El primer `<alumno>` (en orden de documento) |
| `//alumno[nombre="Ana"]` | Alumnos cuyo `<nombre>` es "Ana" |

**Idea:** XPath es al XML lo que una ruta más un pequeño `SELECT` es a una base de datos. Se usa cuando ya hay un DOM cargado y se necesita buscar o filtrar datos sin recorrer el árbol a mano, y para consultas expresivas en validaciones, transformaciones o informes.

## 8. XML con Java

En Java se puede leer (parsear) XML, consultar partes con XPath, modificar y guardar, validar con XSD/DTD y mapear clases Java ↔ XML (binding, JAXB/Jakarta).

| Enfoque | Idea principal | Cuándo usarlo | Pros | Contras |
|---|---|---|---|---|
| DOM (árbol) | Carga todo el XML en memoria como un árbol `Document` | Ficheros pequeños o medianos; leer + modificar + guardar | Sencillo de programar; permite editar | Consume memoria proporcional al tamaño |
| SAX (eventos) | No carga todo el XML: dispara eventos al leerlo secuencialmente | Ficheros grandes; solo lectura | Muy eficiente en memoria | Código más "ceremonioso" si hay lógica compleja |
| Binding (JAXB/Jakarta) | Mapea clases Java ↔ XML (como Gson/Jackson, pero para XML) | Cuando se trabaja con modelos de dominio | Muy productivo; código limpio | Requiere anotar modelos/clases |
| XPath | Lenguaje de consultas sobre un DOM | Buscar/filtrar nodos | Consultas expresivas | Necesita el DOM ya cargado |
| Validación XSD/DTD | Comprobar que el XML cumple un esquema | Intercambio con terceros, contratos formales | Garantiza estructura y tipos | Configuración adicional |

**Dependencias:** para DOM y SAX **no hacen falta** dependencias externas (van en el JDK). Para binding (Java 11+):

```
<dependencies>
  <!-- API JAXB/Jakarta -->
  <dependency>
    <groupId>jakarta.xml.bind</groupId>
    <artifactId>jakarta.xml.bind-api</artifactId>
    <version>4.0.2</version>
  </dependency>
  <!-- Implementación de referencia -->
  <dependency>
    <groupId>org.glassfish.jaxb</groupId>
    <artifactId>jaxb-runtime</artifactId>
    <version>4.0.5</version>
  </dependency>
</dependencies>
```

# 9. Resumen para repasar

| Concepto | Clave |
|---|---|
| Formatos de intercambio | Legibles, independientes del lenguaje y multiplataforma (JSON y XML) |
| JSON | Pares clave-valor; tipos: cadena, número, booleano, null, array, objeto |
| Java y JSON | Sin soporte nativo: `org.json` (simple), Gson (pequeños/medianos), Jackson (empresarial, Spring Boot) |
| `org.json` | `JSONObject`, `JSONArray`, `JSONTokener` (`nextValue()`); `optString` no lanza excepción |
| Gson | `toJson` / `fromJson`; adaptadores para `LocalDate`, etc. |
| Jackson | `ObjectMapper` (`writeValue` / `readValue`), `ObjectWriter`, `TypeReference`; módulo `JavaTimeModule` para fechas |
| Python | Módulo `json` estándar: `dump` / `dumps`, `load` / `loads`, `indent=4` |
| XML | Jerárquico y validable; **bien formado** (sintaxis) vs **válido** (cumple DTD/XSD) |
| XPath | Rutas, predicados `[ ]`, atributos `@`, funciones `text()`, `count()` |
| XML en Java | DOM (árbol), SAX (eventos), binding (JAXB), XPath, validación; DOM y SAX van en el JDK |

# 10. Cosas a tener en cuenta

- El PDF no lista los ficheros de ejemplo de Jackson (la frase "Hay distintos ficheros de ejemplo:" queda sin lista); conviene mirarlos en el aula virtual.
- El mapa mental escribe `DemoOrg.json.java`, mientras que el PDF dice `DemoOrgJson.java`; se ha usado el nombre del PDF.
- El PDF escribe "poom.xml" donde quiere decir `pom.xml`.
- Los conceptos de XML (estructura, bien formado vs válido, XPath) se amplían en los temas de analizadores (1.4) y de consultas XPath (`1_5`, pendiente de recibir).

# Tema 1.4: Analizadores sintácticos (DOM, SAX y binding)

## 1. Qué es un analizador sintáctico (parser)

Para leer, modificar o generar XML de forma estructurada en Java hacen falta **parsers**. Un parser XML lee un documento XML, **comprueba que está bien formado** (etiquetas bien anidadas, una sola raíz...) y permite acceder a su contenido desde Java de forma estructurada.

| Parser | Modelo | Características principales |
|---|---|---|
| **DOM** (Document Object Model) | Basado en **árbol** | Carga todo el XML en memoria y lo representa como un árbol de nodos |
| **SAX** (Simple API for XML) | Basado en **eventos** | No carga todo el documento: lo procesa secuencialmente mientras lo lee |
| **JAXB** (Java Architecture for XML Binding) | Basado en **objetos** | Convierte automáticamente objetos Java ↔ XML (*marshalling* / *unmarshalling*) |

- **DOM:** ideal con XML pequeños cuando hay que leer y modificar libremente.
- **SAX:** ideal para procesar grandes volúmenes de datos sin ocupar mucha memoria.
- **Binding (JAXB):** ideal cuando se trabaja con clases Java que deben guardarse o leerse en XML.

## 2. Modelo DOM

DOM representa el documento XML completo como un **árbol de nodos en memoria** (también se aplica a HTML). Cada etiqueta, atributo o texto se convierte en un **nodo**. Cada documento XML tiene:

1. Un **nodo raíz**, la etiqueta principal (p. ej. `<alumnos>`).
2. **Elementos hijo** (p. ej. `<alumno>`).
3. Cada elemento puede tener **atributos** (como `id="A01"`), **subelementos** (como `<nombre>` y `<edad>`) y **nodos de texto** (como "Ana" o "20").

```xml
<?xml version="1.0" encoding="UTF-8"?>
<alumnos>
    <alumno id="A01">
        <nombre>Ana</nombre>
        <edad>20</edad>
    </alumno>
    <alumno id="A02">
        <nombre>Luis</nombre>
        <edad>22</edad>
    </alumno>
</alumnos>
```

El parser DOM genera en memoria este árbol:

```
Document
 └── Element: alumnos
      ├── Element: alumno (id="A01")
      |    ├── Element: nombre
      |    |    └── Text: "Ana"
      |    └── Element: edad
      |         └── Text: "20"
      └── Element: alumno (id="A02")
           ├── Element: nombre
           |    └── Text: "Luis"
           └── Element: edad
                └── Text: "22"
```

Cada nodo se representa con objetos Java de las clases del paquete `org.w3c.dom` (`Document`, `Element`, `Text`...).

## Cómo funciona DOM

| Fase | Descripción | Clases implicadas |
|---|---|---|
| Creación del parser | Se obtiene un parser DOM; permite configurar opciones (validar el XML, ignorar espacios en blanco...) | `DocumentBuilderFactory` |
| Construcción del árbol | El parser lee el fichero completo y crea un objeto `Document` en memoria con el árbol jerárquico | `DocumentBuilder`, `Document` |

| Fase | Descripción | Clases implicadas |
|---|---|---|
| Normalización | Se eliminan nodos vacíos y se unifican fragmentos de texto adyacentes | `doc.normalize()` |
| Navegación y manipulación | Buscar nodos, recorrer el árbol, acceder a etiquetas, atributos y textos, añadir o borrar elementos | `Node`, `Element`, `NodeList` |
| Escritura (opcional) | Si se modifica el árbol, se puede volver a escribir a disco | `Transformer`, `TransformerFactory` |

DOM es un modelo **bidireccional** (lectura y escritura) y muy flexible, pero consume más memoria y tiempo de carga en XML grandes.

Esquema orientativo de lectura (no está en el PDF; los ejemplos completos están en el aula virtual):

```
DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
DocumentBuilder builder = factory.newDocumentBuilder();
Document doc = builder.parse(new File("alumnos.xml"));
doc.getDocumentElement().normalize();

NodeList alumnos = doc.getElementsByTagName("alumno");
for (int i = 0; i < alumnos.getLength(); i++) {
    Element a = (Element) alumnos.item(i);
    String id = a.getAttribute("id");
    String nombre =
a.getElementsByTagName("nombre").item(0).getTextContent();
}
```

**Ejemplos del aula virtual (DOM):**

| Fichero | Qué hace |
|---|---|
| `LeerXML_DOM.java` | Lee el fichero `alumnos.xml` |
| `GestionBiblioteca_DOM.java` | Lee `biblioteca.xml`, lista los libros, modifica un nodo (año) y un atributo (edición), crea un libro nuevo y escribe `biblioteca_modificada.xml` |
| `GestionBibliotecaCRUD_DOM.java` | Versión de CRUD completo sobre `biblioteca.xml` |

## Métodos más usados

| Clase | Método | Descripción |
|---|---|---|
| `Document` | `getDocumentElement()` | Devuelve el elemento raíz |
| `Document` | `getElementsByTagName(String etiqueta)` | Devuelve todos los elementos con esa etiqueta |
| `Element` | `getTagName()` | Nombre de la etiqueta |
| `Element` | `getAttribute(String nombre)` | Valor de un atributo |
| `Element` | `setAttribute(String nombre, String valor)` | Establece o modifica un atributo |
| `Element` | `getElementsByTagName(String etiqueta)` | Subelementos con esa etiqueta |
| `Element` | `getTextContent()` | Texto contenido en el elemento |
| `Element` | `setTextContent(String texto)` | Cambia el texto de un elemento |
| `Node` | `getNodeName()` | Nombre del nodo |
| `Node` | `getNodeType()` | Tipo de nodo (elemento, texto, comentario...) |
| `Node` | `getParentNode()` | Nodo padre |
| `Node` | `getChildNodes()` | Lista con los nodos hijos |
| `NodeList` | `getLength()` | Número de nodos de la lista |
| `NodeList` | `item(int index)` | Nodo en la posición indicada |
| `DocumentBuilderFactory` | `setIgnoringComments(boolean)` | Ignora comentarios al parsear |
| `DocumentBuilderFactory` | `setIgnoringElementContentWhitespace(boolean)` | Ignora espacios en blanco entre etiquetas |
| `Transformer` | `transform(Source, Result)` | Escribe o exporta el documento a un fichero XML |

`getNodeType()` devuelve un número constante según el tipo de nodo:

| Tipo de nodo | Constante | Valor | Ejemplo en XML |
|---|---|---|---|
| `ELEMENT_NODE` | `Node.ELEMENT_NODE` | 1 | `<alumno>` |
| `ATTRIBUTE_NODE` | `Node.ATTRIBUTE_NODE` | 2 | `id="A01"` |
| `TEXT_NODE` | `Node.TEXT_NODE` | 3 | Ana, 20 |
| `COMMENT_NODE` | `Node.COMMENT_NODE` | 8 | `<!-- comentario -->` |
| `DOCUMENT_NODE` | `Node.DOCUMENT_NODE` | 9 | Todo el documento XML |

# 3. Modelo SAX

SAX es una alternativa a DOM para leer XML de forma **secuencial y basada en eventos**. **No carga todo el XML en memoria**: lo analiza mientras lo lee. Cada vez que el parser encuentra una etiqueta, un texto o el final de un elemento, lanza un **evento** que el programa puede interceptar y procesar.

| Característica | DOM | SAX |
|---|---|---|
| Tipo de modelo | Árbol en memoria | Lectura secuencial por eventos |
| Carga en memoria | Todo el documento | Solo el fragmento actual |
| Acceso a datos | Aleatorio (se puede volver atrás) | Secuencial (solo hacia delante) |
| Modificación | Permite modificar y crear XML | Solo lectura (no modifica) |
| Velocidad y consumo | Más lento, más memoria | Más rápido, menos memoria |
| Ideal para | XML pequeños o editables | XML grandes o lectura rápida |

## Cómo funciona SAX

1. Se crea un parser SAX con `SAXParserFactory`.
2. Se implementa una clase **manejadora** (*handler*) que hereda de `DefaultHandler`.
3. En ella se sobrescriben los métodos que responden a los eventos:
   - `startDocument()`: inicio del documento.
   - `startElement()`: se abre una etiqueta.
   - `characters()`: se encuentra texto dentro de una etiqueta.
   - `endElement()`: se cierra una etiqueta.
   - `endDocument()`: fin del documento.

4. El parser recorre el XML y llama a esos métodos en orden.

| Clase o interfaz | Descripción |
|---|---|
| `SAXParserFactory` | Crea y configura el parser SAX |
| `SAXParser` | Parser que procesa el XML y genera eventos |
| `DefaultHandler` | Clase base que define los métodos de evento |
| `Attributes` | Permite acceder a los atributos de una etiqueta |

(Paquetes `org.xml.sax` y `javax.xml.parsers`.)

Esquema orientativo (no está en el PDF):

```
SAXParser parser = SAXParserFactory.newInstance().newSAXParser();
parser.parse(new File("alumnos.xml"), new DefaultHandler() {
    @Override
    public void startElement(String uri, String localName, String qName,
Attributes attrs) {
        // se abrió <qName>; attrs.getValue("id") da el atributo id
    }
    @Override
    public void characters(char[] ch, int start, int length) {
        // texto dentro de la etiqueta actual: new String(ch, start, length)
    }
    @Override
    public void endElement(String uri, String localName, String qName) {
        // se cerró </qName>
    }
});
```

**Ejemplos del aula virtual (SAX):** Ejemplo 1: `LeerXML_SAX.java` y `ManejadorBiblioteca.java`. Ejemplo 2: crea el manejador y lee `alumnos.xml`. Ejemplo 3: trabaja con objetos `alumno`.

| Ventajas | Desventajas |
|---|---|
| No carga todo el XML en memoria | Solo lectura secuencial, no se puede modificar |
| Rápido y eficiente con documentos grandes | No se puede retroceder ni acceder libremente a etiquetas anteriores |
| Ideal para grandes volúmenes o flujos de datos (logs, importaciones...) | Más difícil de usar en tareas complejas |

| Ventajas | Desventajas |
|---|---|
| No necesita estructuras de datos adicionales | No genera un árbol ni permite guardar cambios |

| Escenario | Recomendación |
|---|---|
| XML grande o *streaming* (muchos registros) | SAX ideal |
| Solo lectura, sin modificar | SAX ideal |
| Procesar por partes o en tiempo real | SAX ideal |
| Necesidad de modificar o guardar XML | Mejor DOM o JAXB |

# 4. Enlace de datos o binding (JAXB)

El **binding** convierte XML directamente en objetos Java (***unmarshalling***) y objetos Java en XML (***marshalling***), sin recorrer etiquetas a mano como con DOM o SAX. Se describe el modelo con **anotaciones** y la librería hace la conversión.

- **Ventajas:** código muy corto y legible; mantenimiento sencillo (clases Java anotadas); ideal para ficheros con estructura estable y un modelo de dominio claro.
- **Limitaciones:** menos flexible para XML "irregulares"; requiere clases Java con **constructor vacío** y anotaciones.

## Dependencias Maven

Hay dos líneas que **no deben mezclarse** en el mismo proyecto:

| Línea | Paquete | Dependencia ( `org.glassfish.jaxb:jaxb-runtime` ) |
|---|---|---|
| **Jakarta JAXB 4.x** (recomendada hoy) | `jakarta.xml.bind.*` | versión `4.0.5` |
| JAXB 2.x (javax) | `javax.xml.bind.*` | versión `2.3.8` |

```
<dependency>
    <groupId>org.glassfish.jaxb</groupId>
    <artifactId>jaxb-runtime</artifactId>
    <version>4.0.5</version>
</dependency>
```

**Por qué hay dos:** JAXB venía dentro del JDK hasta Java 8 ( `javax.xml.bind` ). Desde **Java 11** se quitó del JDK (módulos) y hay que añadirlo como dependencia externa. Además, Oracle donó JAXB a la Eclipse Foundation y el paquete se renombró a `jakarta.xml.bind` (Jakarta EE).

## Idea general

1. Definir el **modelo de dominio** como clases Java (p. ej. `Centro` , `Alumno` ).
2. **Anotar** clases y campos: qué es raíz, elemento o atributo, cómo se representa una lista...
3. Crear un **contexto JAXB** sobre esas clases.
4. **Unmarshalling:** leer XML → obtener objetos poblados.
5. (Opcional) manipular los objetos en Java (filtrar, ordenar, añadir...).
6. **Marshalling:** escribir los objetos de nuevo → XML formateado.

Flujo: **XML** → [unmarshalling] → **objetos Java** → [marshalling] → **XML**.

**Requisitos de las clases:** constructor sin argumentos (**obligatorio**); getters y setters (recomendable, salvo con acceso `FIELD` ); tipos compatibles, o anotar también los tipos compuestos.

Esquema orientativo (no está en el PDF):

```
@XmlRootElement(name = "alumno")
@XmlAccessorType(XmlAccessType.FIELD)
public class Alumno {
    @XmlAttribute(name = "id") private String id;
    @XmlElement(name = "nombre") private String nombre;
    public Alumno() { }                      // constructor vacío
obligatorio
}

JAXBContext ctx = JAXBContext.newInstance(Alumno.class);
Alumno a = (Alumno) ctx.createUnmarshaller().unmarshal(new
File("alumno.xml"));   // XML → objeto
Marshaller m = ctx.createMarshaller();
m.setProperty(Marshaller.JAXB_FORMATTED_OUTPUT, true);
m.marshal(a, new File("alumno_salida.xml"));
// objeto → XML
```

**Ejemplos del aula virtual (JAXB):** `MainLibro.java` (guardar un libro en XML); `MainCurso.java` (relación uno a muchos: un curso con varios alumnos); `MainBiblioteca` (estructura más compleja: una biblioteca con varias obras, cada una con un tipo —enumeración— y un comentario de texto simple).

# Anotaciones más usadas (Jakarta)

| Anotación | Dónde | Descripción | Ejemplo |
|---|---|---|---|
| `@XmlRootElement(name="...")` | Clase | Indica que la clase es la etiqueta **raíz**; `name` define su nombre | `@XmlRootElement(name="alumno")` → `<alumno>...</alumno>` |
| `@XmlElement(name="...")` | Campo o getter | Mapea una propiedad como **subelemento**; sin `name`, usa el nombre del campo | `@XmlElement(name="nombre")` → `<nombre>Lucía</nombre>` |
| `@XmlAttribute(name="...")` | Campo o getter | Mapea un campo como **atributo** XML | `@XmlAttribute(name="id")` → `<alumno id="A01">` |
| `@XmlElementWrapper(name="...")` | Antes de una lista | Agrupa una colección dentro de una etiqueta contenedora; sin ella, los elementos se escriben directamente | `@XmlElementWrapper(name="asignaturas")` + `@XmlElement(name="asignatura")` → `<asignaturas><asignatura>... </asignatura></asignaturas>` |
| `@XmlAccessorType(XmlAccessType.FIELD)` | Clase | Define cómo accede JAXB a los datos: por campos ( `FIELD` ) o por métodos ( `PROPERTY` ); evita anotar cada getter con `FIELD` | JAXB toma todos los campos como elementos XML |
| `@XmlType(propOrder={...})` | Clase | Especifica el **orden** de las etiquetas | `propOrder={"nombre","edad"}` |
| `@XmlTransient` | Campo o método | **Excluye** un campo del XML (no se escribe ni se lee) | `@XmlTransient private String password;` |
| `@XmlValue` | Campo | El valor del objeto es el **contenido de texto** (sin subelementos ni atributos) | `<comentario>Texto libre</comentario>` |
| `@XmlSeeAlso({Clase1.class, Clase2.class})` | Clase base | Indica a JAXB qué otras clases relacionadas considerar (p. ej. subclases) | `@XmlSeeAlso({Alumno.class, Profesor.class})` en `Persona` |
| `@XmlEnum` y `@XmlEnumValue` | Enumeraciones | Define cómo se escriben los valores de un `enum` | `@XmlEnumValue("H") HOMBRE` → `<genero>H</genero>` |

### Ventajas, desventajas y cuándo usarlo

| Ventajas | Desventajas |
|---|---|
| Productividad alta: muy poco código para leer y escribir XML | Menos control fino que DOM/SAX con XML muy irregulares |
| Legibilidad: la estructura del XML se "ve" en las clases | Acoplamiento al modelo: clases bien modeladas y constructor vacío |
| Mantenible: si cambia el XML, se ajustan anotaciones y campos | Curva en casos complejos: listas anidadas, namespaces múltiples o contenido mixto |
| Integración natural: se trabaja con objetos, no con nodos ni eventos | Dependencia externa en Java 11/17+ (no viene en el JDK) |
| Formateo sencillo de la salida con propiedades del *marshaller* | |

**Usar JAXB cuando:** hay un modelo de dominio claro y estable y se quiere importar o exportar datos; se necesita rapidez de desarrollo y código limpio; el XML se parece a "objetos" (árbol razonablemente regular).

**Evitar JAXB cuando:** el XML es enorme y solo se lee de forma secuencial (mejor SAX); se necesitan ediciones finas en el árbol (mejor DOM/XSLT/XPath); el XML es muy irregular o con contenido mixto difícil de mapear a clases.

## 5. Comparativa de enfoques

| Criterio | DOM | SAX | JAXB (binding) |
|---|---|---|---|
| Memoria | Alta (árbol completo) | Baja (*streaming*) | Media (objetos creados) |
| Acceso | Aleatorio (navegable) | Secuencial (solo hacia delante) | Por objetos (modelo de dominio) |
| Edición del XML | Sí (crear/modificar nodos) | No | Indirecta (se editan objetos y se reescribe) |
| Complejidad | Media | Media/alta (eventos) | Baja (anotaciones) |
| Casos ideales | Edición fina, XPath/XSLT | Lecturas masivas, logs | Importación/exportación de datos estructurados |

# 6. Anexo: anotaciones en Java

Una **anotación** añade información o **metadatos** al código. No cambia directamente lo que hace el programa: la interpreta el compilador, la JVM u otras librerías o frameworks (Jakarta, Spring, JUnit...). Se escriben precedidas de `@`:

```
@Override
public String toString() {
    return "Texto";
}
```

`@Override` indica al compilador que se sobrescribe un método de la superclase; si el nombre estuviera mal escrito, avisaría. Las anotaciones existen desde **Java 5 (2004)**, mucho antes de Jakarta: **no pertenecen a Jakarta**, son una característica del lenguaje, y Jakarta, Spring, JUnit, Hibernate, etc. las usan para configurarse sin ficheros XML ni código extra.

**Cómo funcionan:** por sí solas no ejecutan nada; las interpreta otra parte del programa. En JAXB/Jakarta indican cómo convertir una clase Java a XML; en JUnit, `@Test` marca qué métodos son pruebas automáticas.

**Anotaciones básicas del lenguaje (`java.lang` y `java.lang.annotation`):**

| Anotación | Descripción |
|---|---|
| `@Override` | Un método sobrescribe otro de una superclase; el compilador lo verifica |
| `@Deprecated` | Marca un elemento como en desuso; avisa al usarlo |
| `@SuppressWarnings("...")` | El compilador ignora ciertos avisos (`unchecked`, `deprecation`) |
| `@SafeVarargs` | Garantiza que un método con argumentos genéricos variables es seguro (sin *heap pollution*) |
| `@FunctionalInterface` | La interfaz tiene un solo método abstracto (para lambdas) |
| `@Retention` | Hasta cuándo se conserva la anotación (`SOURCE`, `CLASS`, `RUNTIME`) |
| `@Target` | Dónde puede aplicarse (clase, método, parámetro, campo...) |
| `@Inherited` | La anotación se hereda por las subclases |
| `@Documented` | Se incluye en la documentación de javadoc |

| Anotación | Descripción |
|---|---|
| `@Repeatable` (Java 8+) | Permite aplicar la misma anotación varias veces al mismo elemento |
| `@Native` | La constante puede usarse en código nativo (JNI) |

**Anotaciones frecuentes en frameworks:**

| Anotación | Pertenece a | Uso |
|---|---|---|
| `@Test` | JUnit | Marca un método como prueba unitaria |
| `@BeforeEach`, `@AfterEach` | JUnit | Se ejecutan antes o después de cada test |
| `@BeforeAll`, `@AfterAll` | JUnit | Se ejecutan una vez antes o después de todos los tests |
| `@DisplayName` | JUnit 5 | Personaliza el nombre mostrado del test |
| `@Disabled` | JUnit 5 | Desactiva temporalmente una prueba |

# 7. Resumen para repasar

| Concepto | Clave |
|---|---|
| Parser XML | Lee el XML, comprueba que está bien formado y da acceso estructurado |
| DOM | Árbol completo en memoria; lectura **y** escritura; `DocumentBuilderFactory` → `Document`; XML pequeños |
| SAX | Eventos secuenciales; solo lectura; `SAXParserFactory` + `DefaultHandler`; XML grandes |
| Eventos SAX | `startDocument`, `startElement`, `characters`, `endElement`, `endDocument` |
| JAXB | Objetos ↔ XML con anotaciones; unmarshalling (XML → objetos) y marshalling (objetos → XML) |

| Concepto | Clave |
|---|---|
| Requisito JAXB | Constructor vacío y anotaciones; añadir la dependencia desde Java 11 |
| `javax` vs `jakarta` | JAXB 2.x ( `javax.xml.bind` ) vs 4.x ( `jakarta.xml.bind` , recomendada); no mezclarlas |
| Escribir un DOM a disco | `Transformer` / `TransformerFactory` |
| Elegir enfoque | Editar → DOM; leer enorme → SAX; modelo de dominio → JAXB |
| Anotaciones | Metadatos desde Java 5; no son de Jakarta; las interpretan el compilador o las librerías |

## 8. Cosas a tener en cuenta

- Las secciones de este tema se numeran como "4.x" en el PDF (la numeración de los PDFs viene de la unidad completa); aquí se han renumerado dentro del tema.
- El PDF no incluye código completo de DOM, SAX ni JAXB: remite a los ejemplos del aula virtual. Los esquemas marcados como "orientativos" son un apoyo para estudiar, no vienen del temario.
- La validación con DTD/XSD y las consultas XPath con Java se tratan en otros archivos ( `1_5` sobre XPath, pendiente de recibir).

# Tema 1.5: Consultas sobre XML con XPath

## 1. Introducción

**XPath** es un lenguaje de consulta para documentos XML. Permite **localizar nodos** (elementos, atributos, textos...) mediante rutas y filtros. **No modifica: solo selecciona.** Se usa dentro de XSLT, XQuery, validaciones y desde Java (API `javax.xml.xpath` ). Funciona como las rutas de un sistema de archivos, con filtros entre corchetes.

En un documento XML cada parte (elemento, atributo, texto, comentario...) es un **nodo** de un árbol, y XPath recorre ese árbol. Todos los ejemplos del tema usan este catálogo:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!-- Catálogo de productos de ejemplo -->
<catalogo>
    <producto id="P01" categoria="periféricos">
        <nombre>Teclado mecánico</nombre>
        <precio moneda="EUR">49.99</precio>
        <descripcion>Teclado con retroiluminación LED</descripcion>
    </producto>
    <producto id="P02" categoria="periféricos">
        <nombre>Ratón inalámbrico</nombre>
        <precio moneda="EUR">29.50</precio>
        <descripcion>Ratón con sensor óptico de alta precisión</descripcion>
    </producto>
    <producto id="P03" categoria="pantallas">
        <nombre>Monitor 24 pulgadas</nombre>
        <precio moneda="EUR">179.00</precio>
        <descripcion>Resolución Full HD con panel IPS</descripcion>
    </producto>
</catalogo>
```

**Partes de un XML:** elementos ( `<producto>` , `<nombre>` , `<precio>` ), atributos ( `id="P01"` , `categoria="periféricos"` , `moneda="EUR"` ), texto ("Teclado mecánico", "49.99"), comentarios, nodo raíz ( `<catalogo>` ) e instrucción de procesamiento ( `<?xml version="1.0"?>` ).

**Tipos de resultado que puede devolver XPath:**

| Tipo | Qué es |
|---|---|
| `NODE` | Un nodo individual |
| `NODESET` | Un conjunto de nodos (lista) |
| `STRING` | Texto o valor textual |
| `NUMBER` | Un número |
| `BOOLEAN` | Verdadero o falso |

# 2. Rutas y selectores básicos

## Tipos de rutas

| Tipo de ruta | Sintaxis | Ejemplo | Descripción |
|---|---|---|---|
| Absoluta | `/` | `/catalogo/producto/nombre` | Empieza siempre desde el nodo raíz |
| Relativa | (sin `/` al inicio) | `producto/nombre` | Parte del nodo actual, no de la raíz |
| Descendiente | `//` | `//nombre` | Todos los `<nombre>`, sin importar el nivel |
| Con atributo | `@` | `//producto/@id` | Devuelve los valores de un atributo |
| Mixta | `/catalogo//precio` | `/catalogo//precio` | Combina rutas absolutas y descendientes |
| Comodín | `*` o `@*` | `//producto/*` | Todos los elementos o atributos del nodo |

## Selectores y operadores de navegación

| Selector | Significado | Ejemplo | Resultado |
|---|---|---|---|
| `/` | Hijo directo | `/catalogo/producto` | Los `<producto>` directamente bajo `<catalogo>` |
| `//` | Descendiente (cualquier nivel) | `//precio` | Todos los `<precio>` del documento |
| `.` | Nodo actual | `.//nombre` | Busca `<nombre>` desde el nodo actual |
| `..` | Nodo padre | `../@id` | El atributo `id` del elemento padre |

| Selector | Significado | Ejemplo | Resultado |
| --- | --- | --- | --- |
| `@` | Atributo | `//producto/@categoria` | Los valores del atributo `categoria` |
| `*` | Cualquier elemento | `//producto/*` | Todos los hijos de `<producto>` |

## Ejemplos sobre el catálogo

| Consulta XPath | Resultado | Explicación |
|---|---|---|
| `/catalogo/producto/nombre` | "Teclado mecánico", "Ratón inalámbrico", "Monitor 24 pulgadas" | Ruta absoluta desde la raíz |
| `//nombre` | Todos los `<nombre>` | Ruta descendiente, ignora el nivel |
| `//producto/@id` | "P01", "P02", "P03" | Valores de los atributos `id` |
| `/catalogo/producto[1]/nombre` | "Teclado mecánico" | Primer producto del catálogo |
| `(//producto)[last()]/nombre/text()` | "Monitor 24 pulgadas" | Último producto con `last()` |
| `/catalogo/producto[1]/precio` | `<precio moneda="EUR">49.99</precio>` | Nodo `<precio>` completo del primer producto |
| `//producto/*` | nombre, precio, descripcion | Comodín `*` |
| `/catalogo/producto[2]/nombre` | "Ratón inalámbrico" | Nombre del segundo producto |
| `//producto[precio > 100]/nombre` | "Monitor 24 pulgadas" | Productos con precio superior a 100 € |
| `/catalogo//descripcion` | Las tres descripciones | Descripciones a cualquier nivel dentro del catálogo |

## Navegación avanzada: ejes

Los **ejes** son palabras clave que indican la relación entre el nodo actual y los nodos que se quieren seleccionar.

| Eje | Descripción | Ejemplo | Qué devuelve |
|---|---|---|---|
| `child::` | Hijos directos (por defecto) | `child::producto/nombre` | Igual que `producto/nombre` |
| `parent::` | Nodo padre | `parent::catalogo` | El nodo padre del actual |
| `ancestor::` | Antepasados | `ancestor::catalogo` | Todos los nodos superiores hasta la raíz |
| `descendant::` | Descendientes | `descendant::nombre` | Todos los `<nombre>` bajo el actual |
| `following-sibling::` | Hermanos posteriores | `//producto[1]/following-sibling::producto` | Los productos que van después del primero |
| `preceding-sibling::` | Hermanos anteriores | `//producto[3]/preceding-sibling::producto` | Los productos que van antes del tercero |
| `attribute::` | Atributos (equivale a `@`) | `attribute::id` | El atributo `id` |

## Funciones básicas aplicadas a rutas

| Función | Ejemplo | Descripción |
|---|---|---|
| `count()` | `count(//producto)` | Número total de productos (**3**) |
| `sum()` | `sum(//precio)` | Suma de todos los precios (**258.49**) |
| `position()` | `//producto[position()=2]/nombre` | Nombre del segundo producto |
| `last()` | `(//producto)[last()]/nombre` | Nombre del último producto |
| `name()` | `name(//producto[1])` | Nombre del nodo: `producto` |
| `starts-with()` | `//nombre[starts-with(.,'M')]` | Nombres que empiezan por M |
| `contains()` | `//descripcion[contains(.,'LED')]` | Descripciones que contienen "LED" |

# 3. Predicados y filtros

Los **predicados** son condiciones entre corchetes `[ ]` que filtran nodos por su posición o su contenido. Estructura: `/ruta/nodo[condición]`. La condición se evalúa para cada nodo encontrado y solo se devuelven los que la cumplen; se pueden encadenar varios predicados. Ejemplo: `//producto[precio > 50]`.

## Tipos de predicados

| Tipo | Expresión | Resultado |
|---|---|---|
| Por **posición** | `/catalogo/producto[1]` | Primer producto |
| | `(//producto)[last()]` | Último producto |
| | `//producto[position() <= 2]` | Los dos primeros productos |
| Por **valor de un hijo** | `//producto[precio > 100]` | Productos con precio superior a 100 € |
| | `//producto[nombre='Ratón inalámbrico']` | Producto cuyo nombre es exactamente ese |
| Por **valor de un atributo** (`@atributo`) | `//producto[@categoria='periféricos']` | Productos de la categoría "periféricos" |
| | `//producto[@id='P02']/nombre` | Nombre del producto con id "P02" |
| | `//producto[@id!='P01']` | Todos los productos excepto el de id P01 |
| **Combinados** (`and`, `or`, `not()`) | `//producto[@categoria='periféricos' and precio < 50]` | Periféricos con precio menor que 50 € |
| | `//producto[@categoria='pantallas' or precio > 100]` | Pantallas o con precio superior a 100 € |
| | `//producto[not(@categoria='pantallas')]` | Todos los que no son pantallas |

`position()` devuelve el número de posición del nodo dentro del conjunto actual.

## Funciones en los filtros

| Función | Ejemplo | Resultado |
|---|---|---|
| `contains(c1, c2)` | `//producto[contains(nombre, 'Ratón')]` | Productos cuyo nombre contiene "Ratón" |
| `starts-with(c1, c2)` | `//producto[starts-with(nombre, 'M')]` | Nombre que empieza por "M" |
| `ends-with(c1, c2)` (solo XPath 2.0) | `//producto[ends-with(nombre, 'co')]` | Nombre que termina en "co" |
| `string-length()` | `//descripcion[string-length(.) > 30]` | Descripciones de más de 30 caracteres |
| `normalize-space()` | `//producto[normalize-space(nombre)='Teclado mecánico']` | Elimina espacios extra antes de comparar |

**Predicados encadenados** (refinan la búsqueda paso a paso). Periféricos con precio menor que 50 y nombre que contiene "Ratón":

```
//producto[@categoria='periféricos'][precio < 50][contains(nombre, 'Ratón')]
```

**Evaluaciones booleanas:** `count(//producto[precio > 100]) > 0` es `true` si hay al menos un producto con precio > 100; `count(//producto[@categoria='pantallas']) = 1` es `true` si solo existe una pantalla.

## Ejemplos con resultado

| Consulta XPath | Resultado | Explicación |
|---|---|---|
| `//producto[@id='P02']/nombre/text()` | Ratón inalámbrico | Nombre del producto con id P02 |
| `//producto[precio > 50]/nombre/text()` | Monitor 24 pulgadas | Precio superior a 50 € |
| `//producto[@categoria='periféricos']/nombre/text()` | Teclado mecánico, Ratón inalámbrico | Filtra por atributo `categoria` |

| Consulta XPath | Resultado | Explicación |
|---|---|---|
| `(//producto)[1]/nombre/text()` | Teclado mecánico | Primer producto |
| `(//producto)[last()]/nombre/text()` | Monitor 24 pulgadas | Último producto |
| `//producto[position()=2]/precio/text()` | 29.50 | Precio del segundo producto |
| `//producto[precio < 50]/@id` | P01, P02 | Ids de productos con precio < 50 € |
| `//producto[precio < 50 and contains(nombre,'Ratón')]/nombre/text()` | Ratón inalámbrico | Texto y valor numérico a la vez |
| `//producto[starts-with(nombre,'M')]/nombre/text()` | Monitor 24 pulgadas | Nombre que empieza por M |
| `//producto[contains(descripcion,'LED')]/nombre/text()` | Teclado mecánico | Descripción que contiene "LED" |
| `//producto[not(@categoria='pantallas')]/nombre/text()` | Teclado mecánico, Ratón inalámbrico | Excluye las pantallas |
| `//producto[@categoria='periféricos'][precio < 50]/nombre/text()` | Teclado mecánico, Ratón inalámbrico | Filtro doble |
| `//producto[@id!='P01']/nombre/text()` | Ratón inalámbrico, Monitor 24 pulgadas | Todos menos P01 |
| `//producto[precio>20 and precio<100]/nombre/text()` | Teclado mecánico, Ratón inalámbrico | Rango de precios |
| `//producto[precio > 50 or contains(descripcion,'Full HD')]/nombre/text()` | Monitor 24 pulgadas | Condición compuesta con `or` |
| `//producto[normalize-space(nombre)='Teclado mecánico']/@id` | P01 | Normaliza espacios antes de comparar |

| Consulta XPath | Resultado | Explicación |
|---|---|---|
| `//producto[precio = max(//precio)]/nombre/text()` (solo XPath 2.0) | Monitor 24 pulgadas | El producto más caro |
| `count(//producto[@categoria='periféricos'])` | 2 | Número de periféricos |
| `sum(//precio)` | 258.49 | Suma de los precios |

Tres ejemplos del PDF dan un resultado distinto del real (se han comprobado con un evaluador XPath 1.0); los valores correctos son:

| Consulta XPath | Resultado correcto (el PDF indica otro) |
|---|---|
| `//producto[precio > 30 and @categoria='periféricos']/nombre/text()` | **Teclado mecánico** (49.99 > 30). El PDF dice "ninguno" |
| `//producto[string-length(descripcion) > 30]/nombre/text()` | **Las tres** descripciones miden más de 30 caracteres (32, 41 y 32). El PDF solo lista dos |
| `//producto[string-length(nombre) > 10]/nombre/text()` | **Los tres** nombres miden más de 10 caracteres. El PDF solo lista dos |

### Buenas prácticas

- Se pueden usar comillas simples o dobles ( `[@id='P01']` o `[@id="P01"]` ).
- Las comparaciones **distinguen mayúsculas y minúsculas**.
- Si puede haber espacios sobrantes, usar `normalize-space()` antes de comparar.
- Evitar combinaciones ambiguas: `(//producto)[1]` y `//producto[1]` **no son equivalentes**:
  - `(//producto)[1]` → el primer producto de todo el documento.
  - `//producto[1]` → el primer producto de cada grupo de hermanos bajo su padre.

## 4. Funciones XPath más utilizadas

Las funciones permiten buscar, filtrar y manipular resultados sin modificar el XML. Se pueden usar dentro de predicados `[ ]` o en el cuerpo de la expresión ( `sum(//precio)` devuelve un número; `//producto[precio > sum(//precio) div 3]` la usa dentro del filtro).

| Tipo de función | Funciones más comunes |
|---|---|
| Numéricas | `count()`, `sum()`, `floor()`, `ceiling()`, `round()` |
| De cadena de texto | `string()`, `concat()`, `contains()`, `starts-with()`, `string-length()`, `normalize-space()` |
| De posición y contexto | `position()`, `last()`, `name()` |
| Booleanas | `not()`, `boolean()`, operadores `and`, `or` |
| De nodo | `local-name()`, `namespace-uri()`, `name()` |

## Numéricas

| Función | Ejemplo | Resultado | Explicación |
|---|---|---|---|
| `count()` | `count(//producto)` | 3 | Número total de productos |
| `sum()` | `sum(//precio)` | 258.49 | Suma de los `<precio>` |
| `floor()` | `floor(//precio[1])` | 49 | Redondea hacia abajo el primer precio |
| `ceiling()` | `ceiling((//precio)[2])` | 30 | Redondea hacia arriba el segundo precio (29.50 → 30) |
| `round()` | `round(sum(//precio) div count(//precio))` | 86 | Precio medio redondeado |

## De cadena de texto

| Función | Ejemplo | Resultado | Explicación |
|---|---|---|---|
| `string()` | `string(//producto[@id='P01']/nombre)` | Teclado mecánico | Convierte el nodo a texto |
| `concat()` | `concat(//producto[1]/nombre, ' - ', //producto[1]/precio)` | Teclado mecánico - 49.99 | Une varios textos o valores |
| `contains()` | `//producto[contains(descripcion,'LED')]/nombre` | Teclado mecánico | Si una cadena contiene otra |
| `starts-with()` | `//producto[starts-with(nombre,'R')]/nombre` | Ratón inalámbrico | Nombre que empieza por "R" |

| Función | Ejemplo | Resultado | Explicación |
|---|---|---|---|
| `string-length()` | `string-length(//descripcion[1])` | 32 | Número de caracteres del texto |
| `normalize-space()` | `//producto[normalize-space(nombre)='Teclado mecánico']/@id` | P01 | Quita espacios sobrantes antes de comparar |

## De posición y contexto

| Función | Ejemplo | Resultado | Explicación |
|---|---|---|---|
| `position()` | `//producto[position()=2]/nombre/text()` | Ratón inalámbrico | Nodo en la posición 2 |
| `last()` | `(//producto)[last()]/@id` | P03 | Último producto del documento |
| `name()` | `name(//producto[1])` | producto | Nombre del nodo (etiqueta) |
| `local-name()` | `local-name(//precio[1])` | precio | Como `name()`, sin el prefijo del espacio de nombres |

## Booleanas y de comparación

| Función / operador | Ejemplo | Resultado | Explicación |
|---|---|---|---|
| `boolean()` | `boolean(//producto[@id='P01'])` | true | `true` si el nodo existe |
| `not()` | `not(//producto[@id='P05'])` | true | `true` si el nodo no existe |

| Función / operador | Ejemplo | Resultado | Explicación |
|---|---|---|---|
| `and` | `//producto[@categoria='periféricos' and precio < 50]` | Teclado mecánico, Ratón inalámbrico | Cumplen ambas condiciones |
| `or` | `//producto[@categoria='pantallas' or precio > 100]` | Monitor 24 pulgadas | Cumplen al menos una |
| `=` | `//producto[precio = 49.99]/@id` | P01 | Compara valores exactos |
| `!=` | `//producto[@id!='P03']/nombre` | Teclado mecánico, Ratón inalámbrico | Distinto de |

## Funciones combinadas

| Ejemplo | Resultado | Descripción |
|---|---|---|
| `round(sum(//precio) div count(//precio))` | 86 | Precio medio redondeado |
| `//producto[starts-with(nombre,'M') and precio > 100]/nombre` | Monitor 24 pulgadas | Texto y número en la misma condición |
| `count(//producto[contains(descripcion,'sensor')])` | 1 | Productos cuya descripción incluye "sensor" |

Otra combinación del PDF, `//producto[string-length(nombre) > 10]/nombre`, devuelve los **tres** productos (ver la tabla de correcciones de arriba).

**XPath 1.0 vs 2.0:** XPath 1.0 solo admite funciones básicas; XPath 2.0 y superiores añaden `upper-case()`, `lower-case()`, `ends-with()`, `avg()`, `max()`, `min()`... La API estándar de Java SE solo soporta XPath 1.0.

# 5. Uso de XPath desde Java

La API estándar `javax.xml.xpath` evalúa expresiones XPath sobre un árbol DOM, sin recorrer el XML nodo a nodo. Sirve para localizar nodos y valores, contar, filtrar y extraer datos **sin modificar el archivo**, y para automatizar análisis, búsquedas o validaciones.

## Clases principales

| Clase / interfaz | Paquete | Función |
|---|---|---|
| `DocumentBuilderFactory` | `javax.xml.parsers` | Crea un analizador DOM que carga el XML en memoria |
| `Document` | `org.w3c.dom` | Representa el árbol completo del XML |
| `XPathFactory` | `javax.xml.xpath` | Crea objetos XPath que evalúan las expresiones |
| `XPath` | `javax.xml.xpath` | Objeto principal para compilar y ejecutar expresiones |
| `XPathExpression` | `javax.xml.xpath` | Compila una expresión para reutilizarla varias veces |
| `XPathConstants` | `javax.xml.xpath` | Tipos de resultado posibles ( `STRING` , `NUMBER` , `BOOLEAN` , `NODESET` ) |

## Flujo general

1. **Cargar el XML** con `DocumentBuilderFactory` → objeto `Document` .
2. **Crear el XPath:** `XPathFactory.newInstance().newXPath()` .
3. **Compilar o evaluar** la expresión, escrita como cadena (p. ej. `//producto[precio>50]/nombre/text()` ).
4. **Obtener el resultado** según el tipo indicado con `XPathConstants` .

| Tipo de resultado | Constante | Tipo en Java | Ejemplo | Resultado |
|---|---|---|---|---|
| Cadena | `XPathConstants.STRING` | `String` | `(//producto)[1]/nombre/text()` | "Teclado mecánico" |
| Número | `XPathConstants.NUMBER` | `Double` | `count(//producto)` | 3.0 |
| Booleano | `XPathConstants.BOOLEAN` | `Boolean` | `count(//producto[precio>100])>0` | true / false |

| Tipo de resultado | Constante | Tipo en Java | Ejemplo | Resultado |
|---|---|---|---|---|
| Conjunto de nodos | `XPathConstants.NODESET` | `NodeList` | `//producto/nombre/text()` | Lista de nodos |

| Tipo de consulta | Ejemplo XPath | Tipo de resultado |
|---|---|---|
| Seleccionar nodos | `//producto/nombre/text()` | `NODESET` |
| Contar nodos | `count(//producto)` | `NUMBER` |
| Sumar valores | `sum(//precio)` | `NUMBER` |
| Buscar texto | `//producto[contains(descripcion,'LED')]` | `NODESET` |
| Comprobar existencia | `count(//producto[precio>100])>0` | `BOOLEAN` |
| Extraer un valor concreto | `(//producto)[1]/@id` | `STRING` |

## Evaluar expresiones

La evaluación se hace con `xpath.evaluate(String expression, Object item, QName returnType)`. El método devuelve un `Object` que hay que convertir (*cast*) al tipo correspondiente.

| Parámetro | Tipo | Descripción |
|---|---|---|
| `expression` | `String` | La expresión XPath, escrita como texto |
| `item` | `Object` | Nodo o documento sobre el que se ejecuta: normalmente el `Document`, o un `Element` concreto para aplicarla sobre una parte |
| `returnType` | `QName` | Tipo de resultado, con una constante de `XPathConstants` |

**Contexto de evaluación:** si `item` es el `Document`, la consulta se evalúa desde la raíz; si es un nodo (`Element producto`), se evalúa desde ese nodo. Por ejemplo, `xpath.evaluate("precio/text()", producto, STRING)` devuelve el precio del producto actual dentro de un bucle.

**Compilar y reutilizar:** si la expresión se ejecuta muchas veces (p. ej. en un bucle), es más eficiente compilarla una vez con `XPathExpression`; evita analizarla cada vez y mejora el rendimiento con mucho XML.

```
XPathExpression expr = xpath.compile("//producto[precio >
50]/nombre/text()");
NodeList lista = (NodeList) expr.evaluate(doc, XPathConstants.NODESET);
```

Esquema completo (no está en el PDF; se ha compilado y ejecutado sobre el catálogo del tema):

```
Document doc = DocumentBuilderFactory.newInstance()
        .newDocumentBuilder().parse(new File("catalogo.xml"));
XPath xpath = XPathFactory.newInstance().newXPath();

NodeList caros = (NodeList) xpath.evaluate(
        "//producto[precio > 50]/nombre/text()", doc,
XPathConstants.NODESET);
for (int i = 0; i < caros.getLength(); i++) {
    System.out.println("Caro: " + caros.item(i).getNodeValue());
}
String primero = (String) xpath.evaluate("(//producto)[1]/nombre/text()",
doc, XPathConstants.STRING);
Double suma = (Double) xpath.evaluate("sum(//precio)", doc,
XPathConstants.NUMBER);
Boolean hayPantallas = (Boolean) xpath.evaluate(
        "count(//producto[@categoria='pantallas']) > 0", doc,
XPathConstants.BOOLEAN);

// Contexto: evaluar desde un nodo concreto dentro de un bucle
NodeList productos = (NodeList) xpath.evaluate("//producto", doc,
XPathConstants.NODESET);
XPathExpression precio = xpath.compile("precio/text()");
for (int i = 0; i < productos.getLength(); i++) {
    Element p = (Element) productos.item(i);
    System.out.println(p.getAttribute("id") + " -> " + precio.evaluate(p));
}
```

XPath 1.0 solo devuelve esos cuatro tipos básicos; XPath 2.0 añade secuencias, fechas y otros, pero no están disponibles en la API estándar de Java SE.

### Excepciones comunes

| Excepción | Causa habitual | Solución |
|---|---|---|
| `XPathExpressionException` | Error de sintaxis en la expresión o referencia a nodos inexistentes | Revisar la expresión (comillas, paréntesis, funciones) |
| `ClassCastException` | Se convierte el resultado a un tipo incorrecto | Comprobar que el `returnType` coincide con el dato esperado |
| `NullPointerException` | Se evalúa sobre un documento o nodo no inicializado | Comprobar que el documento se cargó bien |

## 6. Resumen para repasar

| Concepto | Clave |
|---|---|
| XPath | Lenguaje para **seleccionar** nodos en un XML (no modifica) |
| Rutas | `/` absoluta, sin `/` relativa, `//` descendiente, `@` atributo, `*` comodín, `.` actual, `..` padre |
| Predicados | `[ ]` por posición, por valor de hijo, por atributo, combinados con `and` / `or` / `not()` |
| `(//producto)[1]` vs `//producto[1]` | El primero del documento vs el primero de cada grupo de hermanos |
| Funciones | Numéricas, de cadena, de posición, booleanas, de nodo; `ends-with`, `max`, `avg` son XPath 2.0 |
| Ejes | `child::`, `parent::`, `ancestor::`, `descendant::`, `following-sibling::`, `preceding-sibling::`, `attribute::` |
| En Java | `DocumentBuilderFactory` → `Document`; `XPathFactory` → `XPath`; `XPath.evaluate(expr, item, tipo)` |
| Tipos de resultado | `STRING` → `String`, `NUMBER` → `Double`, `BOOLEAN` → `Boolean`, `NODESET` → `NodeList` |

| Concepto | Clave |
|---|---|
| Rendimiento | Compilar con `XPathExpression` si se repite la consulta |
| Ventaja | Una consulta precisa en vez de bucles y condiciones sobre el DOM |

## 7. Cosas a tener en cuenta

- **Errores en las tablas del PDF:** además de los tres ejemplos corregidos arriba, `string-length(//descripcion[1])` da **32** y no 35, y `ceiling(//precio[2])` no devuelve 30: `//precio[2]` busca el *segundo* `<precio>` *dentro de cada producto* y no existe, así que el resultado es `NaN`. Para el segundo precio del documento hay que escribir `ceiling((//precio)[2])` (es el caso de `(//producto)[1]` frente a `//producto[1]`).
- `//producto[1]/../@*` no devuelve nada porque `<catalogo>` no tiene atributos.
- El esquema de código Java de la sección 5 no viene del temario; se ha comprobado que compila y funciona con el catálogo del tema.
- Los ejemplos de XPath con DOM del aula virtual ( `XmlToJsonXPath_v2` ) se retoman en el tema 1.6 de conversión.

# Tema 1.6: Conversión entre formatos (XML ↔ JSON)

## 1. Introducción

Las aplicaciones necesitan intercambiar información y no siempre usan el mismo formato, así que hay que **convertir entre formatos**. Los dos más extendidos para datos estructurados son XML y JSON; ambos representan datos jerárquicos, con enfoques distintos:

- **XML:** usa etiquetas para definir la estructura (como HTML). Muy versátil, extensible y autodescriptivo, pero más verboso (más caracteres y más espacio).
- **JSON:** se basa en pares clave–valor, con una estructura más ligera y una sintaxis cercana a los objetos de programación. Más compacto y fácil de leer y procesar en JavaScript, Python o Java.

**Cuándo hace falta convertir:**

- Un servicio web antiguo que aún envía XML y debe integrarse con una aplicación moderna que usa JSON.

- Una base de datos o sistema de informes que solo admite XML, cuando los datos llegan de una API REST que devuelve JSON.
- Migraciones o integraciones de sistemas que deben mantener compatibilidad con varios formatos.

### XML vs JSON

| Característica | XML | JSON |
|---|---|---|
| Estructura | Jerárquica basada en etiquetas con atributos y contenido | Jerárquica basada en objetos, arrays y pares clave–valor |
| Legibilidad | Clara, pero más extensa | Muy legible y concisa |
| Tamaño del fichero | Mayor, por las etiquetas | Más pequeño y eficiente |
| Soporte en herramientas empresariales | Amplio: validaciones (DTD/XSD), transformaciones (XSLT)... | Ideal para aplicaciones web y APIs REST |
| Tipos de datos | Todo el contenido es texto (requiere *parseo* posterior) | Tipos nativos: números, booleanos, `null` |
| Metadatos | Permite atributos y comentarios | No admite comentarios ni atributos |
| Procesamiento en Java | DOM, SAX o StAX | Gson o Jackson |

**Transformaciones que se estudian:**

1. **XML → JSON:** leer un XML, recorrer su estructura y generar un texto JSON equivalente.
2. **JSON → XML:** leer un JSON y crear un documento XML con la misma jerarquía.

Además, las diferencias estructurales y las precauciones para **no perder información** al convertir (atributos, arrays, valores nulos).

## 2. Enfoques de conversión

Hay dos: **conversión manual** y **conversión automática mediante librerías**.

## Conversión manual

Consiste en procesar explícitamente el formato de origen, extrayendo los datos con las clases del API estándar de Java (DOM, SAX o StAX) y reconstruyendo a mano el formato de destino con ayuda de librerías como Gson o Jackson. Da **control total**, pero requiere más código y entender ambos formatos.

Proceso general de XML → JSON, en tres etapas:

| Etapa | Descripción | Tecnologías |
|---|---|---|
| 1. Lectura del documento XML | Se carga el XML en memoria y se analiza su estructura | DOM, SAX |
| 2. Conversión a una estructura intermedia | Se recorren los nodos y se crean objetos Java, mapas o listas con los datos | Clases de modelo ( `Cliente` , `Pedido` ...), `HashMap` , `ArrayList` |
| 3. Serialización a JSON | Una librería convierte la estructura Java en texto JSON | `Gson.toJson()` , `ObjectMapper.writeValueAsString()` |

El proceso inverso (JSON → XML) sigue la misma lógica en sentido contrario.

| Ventajas | Inconvenientes |
|---|---|
| Control total (atributos, listas, nulos) | Más código y complejidad |
| Se pueden aplicar reglas de negocio o transformaciones personalizadas | Propenso a errores si el XML es complejo |
| No requiere librerías externas si solo se usa DOM y clases básicas | Más mantenimiento cuando cambian los formatos |

**Ejemplos del aula virtual:**

- `XmlToJsonManual` : usa **DOM** para leer y recorrer el XML y extrae los valores con `getElementsByTagName()` y `getAttribute()` .
- `XmlToJsonXPath_v2` : usa **XPath** con expresiones como `"nombre/text()"` o `"number(edad)"` , sin recorrer a mano el árbol DOM: código más conciso y declarativo.
- `JsonToXmlManual` : pasa un fichero JSON a XML.

### Conversión automática mediante librerías

Varias librerías convierten directamente entre XML y JSON sin recorrer los nodos:

| Librería | Descripción | Ejemplo de uso |
|---|---|---|
| `org.json` (paquete clásico) | Incluye `JSONObject` y la clase `XML`, con métodos para convertir entre formatos | `JSONObject json = XML.toJSONObject(xmlString);` |
| Jackson DataFormat XML | Extensión de Jackson que lee y escribe XML como si fuera JSON | Usa `XmlMapper` (subclase de `ObjectMapper`) |
| Gson + adaptadores personalizados | Genera JSON desde objetos Java obtenidos del XML (requiere un paso intermedio) | Conversión en dos pasos: XML → objeto → JSON |

| Ventajas | Inconvenientes |
|---|---|
| Rápido y sencillo de implementar | Menor control sobre la estructura final: la librería decide cómo representar atributos o listas |
| Poco código, menos errores humanos | Puede perder información si el origen tiene elementos difíciles de mapear (atributos, comentarios, nodos vacíos) |
| Ideal para prototipos, integraciones rápidas o servicios web | Algunas librerías no manejan bien los tipos mixtos (texto + subnodos) |

## 3. Esquema general de conversiones

| Tipo | Sentido | Tecnología base | Librerías | Método / enfoque | Ejemplo del proyecto |
|---|---|---|---|---|---|
| **Manual** (proyecto `conversion-manual.zip`) | XML → JSON | DOM | `javax.xml.parsers`, `org.w3c.dom`, Gson | Recorrer el árbol DOM y construir `List<Map<String,Object>>`; serializar con Gson | `XmlToJsonManual1.java` |
| Manual | XML → JSON | XPath + DOM | `javax.xml.xpath`, `org.w3c.dom`, Gson | Consultas XPath sobre DOM para extraer datos y mapearlos a una lista de mapas | `XmlToJsonManual2.java` |

| Tipo | Sentido | Tecnología base | Librerías | Método / enfoque | Ejemplo del proyecto |
|---|---|---|---|---|---|
| Manual | XML → JSON | SAX (por eventos) | `javax.xml.parsers.SAXParser`, `DefaultHandler`, Gson | Procesamiento por eventos (inicio/fin de etiqueta, texto); el *handler* construye una lista de clientes | `XmlToJsonManual3.java` + `ClienteHandler.java` |
| Manual | JSON → XML | DOM + Gson | `javax.xml.parsers`, `org.w3c.dom`, Gson | Leer JSON con Gson, convertir a `List<Map<String,Object>>` y generar el DOM elemento a elemento | `JsonToXmlManual1.java` |
| Manual | JSON → XML | DOM + Gson (recursivo) | `javax.xml.parsers`, `org.w3c.dom`, Gson | Conversión recursiva: maneja objetos anidados y listas JSON complejas dentro del DOM | `JsonToXmlManual2_Recursivo.java` |
| **Automática** (proyecto `conversion-automatica.zip`) | XML → JSON | Binding (JAXB) + Gson | `com.google.gson.Gson`, `jakarta.xml.bind` | JAXB convierte el XML en objetos Java según las anotaciones del modelo; Gson serializa esos objetos a JSON legible | `XmlToJsonAutomatica1.java` y clases del modelo |
| Automática | JSON → XML | Gson + JAXB | `com.google.gson.Gson`, `jakarta.xml.bind` | Gson: JSON → objetos Java (`List<Cliente>` o `Clientes`); JAXB: objetos → XML (*marshal*) | `JsonToXmlAutomatico1.java` y clases del modelo |

# 4. Resumen para repasar

| Concepto | Clave |
|---|---|
| Por qué convertir | Integrar sistemas que usan formatos distintos (servicios antiguos en XML, APIs REST en JSON) |
| XML vs JSON | XML: etiquetas, atributos, comentarios, validación, más verboso. JSON: ligero, tipos nativos, sin atributos ni comentarios |
| Conversión manual | DOM/SAX/StAX (o XPath) para leer + estructura intermedia (mapas/objetos) + Gson/Jackson para escribir; control total pero más código |
| Conversión automática | `org.json` (`XML.toJSONObject`), Jackson `XmlMapper`, JAXB + Gson; rápida pero con menos control |
| Riesgo | Pérdida de información: atributos, comentarios, nodos vacíos, tipos mixtos, arrays de un solo elemento |
| Dirección inversa | Misma lógica en sentido contrario (JSON → estructura intermedia → DOM/JAXB) |

## 5. Cosas a tener en cuenta

- En la primera fila de la tabla de esquema, el PDF mezcla en la celda del ejemplo el nombre del fichero con parte de la explicación de la fila siguiente ("XmlToJConsultas XPath... mapas.sonManual1.java"); lo que quiere decir es `XmlToJsonManual1.java`.
- El PDF habla de las diferencias estructurales y de las precauciones para no perder información, pero no las desarrolla; los ejemplos del aula virtual (`conversion-manual.zip` y `conversion-automatica.zip`) son el material práctico.
- Los apartados "4. XML con Java" y "JSON con Java" de los temas 1.3 y 1.4 son la base de estos ejemplos: DOM, SAX y JAXB para XML, y Gson o Jackson para JSON.

# Anexo 1.1: La clase `RandomAccessFile`

Complementa el apartado de acceso aleatorio del tema 1.1.

## 1. Qué es

- Sirve para **leer y escribir en cualquier posición** de un fichero.
- Está en `java.io` y permite acceso aleatorio: no hace falta leer todo el fichero para llegar a una parte concreta.

```
RandomAccessFile raf = new RandomAccessFile("archivo.dat", "r");    // solo
lectura
RandomAccessFile raf = new RandomAccessFile("archivo.dat", "rw");   //
lectura y escritura
```

`"rw"` crea el fichero si no existe.

## 2. Métodos principales

**Posicionamiento:**

| Método | Qué hace |
|---|---|
| `seek(long pos)` | Mueve el puntero a la posición `pos` (en bytes, desde el inicio). `raf.seek(0)` va al inicio; `raf.seek(raf.length())` va al final para añadir |
| `getFilePointer()` | Devuelve la posición actual del puntero |
| `length()` | Tamaño del fichero en bytes |

| Método | Qué hace |
|---|---|
| `setLength(long newLength)` | Cambia el tamaño: si es menor que `length()` **trunca** (borra bytes del final); si es mayor, **añade bytes vacíos** al final |

**Escritura** (en **binario**, no como texto plano): `writeInt(int)`, `writeDouble(double)`, `writeBoolean(boolean)`, `writeChar(int)` y `writeUTF(String)` (longitud + caracteres en UTF-8).

`raf.writeInt(123)` no escribe "123" sino **4 bytes** binarios que representan ese número; abierto en un editor de texto se ven símbolos raros. `writeUTF("Pedro")` guarda `[2 bytes de longitud][bytes de cada carácter]`.

```
raf.writeInt(123);
raf.writeUTF("Hola");
raf.writeDouble(9.75);
```

| Método | Qué escribe | Tamaño | ¿Legible en el Bloc de notas? |
|---|---|---|---|
| `writeInt(123)` | El número 123 en binario fijo | 4 bytes | No |
| `writeUTF("123")` | Longitud (2 bytes) + caracteres `1 2 3` | 5 bytes | Parcial (se ve "123", pero hay bytes ocultos delante) |
| `FileWriter.write("123")` | Solo los caracteres | 3 bytes | Sí |

**Lectura:** `readInt()`, `readDouble()`, `readBoolean()`, `readChar()`, `readUTF()`. Hay que leer **en el mismo orden y con el mismo tipo** con que se escribió; si no, salen datos corruptos.

```
int num = raf.readInt();
String txt = raf.readUTF();
double nota = raf.readDouble();
```

**Cierre:** `close()` cierra el fichero y libera recursos; con try-with-resources no hace falta llamarlo.

### Tamaño de cada tipo

| Tipo | Bytes | Bits | Comentario |
|---|---|---|---|
| `byte` | 1 | 8 | Entero con signo (-128 a 127) |
| `short` | 2 | 16 | Entero con signo |
| `char` | 2 | 16 | Carácter Unicode (UTF-16) |
| `int` | 4 | 32 | Entero con signo |
| `long` | 8 | 64 | Entero con signo |
| `float` | 4 | 32 | Decimal de precisión simple (IEEE 754) |
| `double` | 8 | 64 | Decimal de precisión doble (IEEE 754) |
| `boolean` | 1* | — | En RAM no está estandarizado, pero `writeBoolean` ocupa 1 byte (0 o 1) |

## 3. Cuándo usarla

Cuando se quieren guardar **registros binarios** y poder ir directamente a uno sin leer todo el fichero. Ejemplo clásico: guardar empleados ( `int id` , `String nombre` , `double salario` ) y saltar al 3.er con `seek(posicion)` para leer solo ese registro. Es como una **base de datos primitiva en fichero**, no un fichero de texto.

## 4. Registros de tamaño fijo (ejemplo de apoyo)

El PDF dice que se puede saltar al 3.er empleado con `seek()` , pero hay una condición que no menciona: como `writeUTF` ocupa un número **variable** de bytes (según la longitud de la cadena), para calcular la posición de un registro todos deben medir lo mismo. Una forma es escribir el nombre con un número fijo de caracteres ( `writeChars` , 2 bytes por carácter). Este ejemplo no viene del temario; se ha compilado y ejecutado.

```
static final int LONG_NOMBRE = 20;                          // caracteres
fijos por nombre
static final int TAM_REGISTRO = 4 + LONG_NOMBRE * 2 + 8;    // int + 20 chars
+ double = 52 bytes

static void escribir(RandomAccessFile raf, int id, String nombre, double
salario)
        throws IOException {
    StringBuilder sb = new StringBuilder(nombre);
```

```
    sb.setLength(LONG_NOMBRE);                  // recorta o rellena (con
'\0')
    raf.writeInt(id);
    raf.writeChars(sb.toString());              // 2 bytes por carácter
    raf.writeDouble(salario);
}

static void leer(RandomAccessFile raf) throws IOException {
    int id = raf.readInt();
    StringBuilder nombre = new StringBuilder();
    for (int i = 0; i < LONG_NOMBRE; i++) nombre.append(raf.readChar());
    double salario = raf.readDouble();
    System.out.println(id + " | " + nombre.toString().trim() + " | " +
salario);
}

// Uso
raf.seek(2L * TAM_REGISTRO);                    // saltar directamente al
3.er registro
leer(raf);                                      // 3 | Marta | 2100.75
raf.seek(TAM_REGISTRO + 4 + LONG_NOMBRE * 2);   // salario del 2.º registro
raf.writeDouble(1999.99);                       // modificarlo sin tocar el
resto
```

# 5. Resumen para repasar

| Concepto | Clave |
|---|---|
| Modos | `"r"` solo lectura; `"rw"` lectura y escritura (crea el fichero si no existe) |
| Puntero | `seek(pos)`, `getFilePointer()`, `length()`, `setLength()` |
| Escritura | Binaria: `writeInt` (4 bytes), `writeDouble` (8), `writeBoolean` (1), `writeChar` (2), `writeUTF` (2 + caracteres) |
| Lectura | En el mismo orden y tipo que la escritura |
| Ideal para | Registros binarios de tamaño fijo con acceso directo ("mini base de datos") |

# Anexo 1.2: Ejemplos de `RandomAccessFile` y bytes en disco

Tres ficheros del aula virtual que ilustran el anexo 1.1: `DemoBytes.java`, `Ejemplo2RandomAccess.java` y un `datos.dat` generado por la versión anterior. Los dos programas se han compilado y ejecutado con Java 21 y los resultados de abajo son los reales.

## 1. `DemoBytes.java`: qué bytes escribe cada método

Escribe el número 123 de tres formas y muestra los bytes en hexadecimal con el método auxiliar `mostrarBytes`, que lee el fichero con `Files.readAllBytes` y imprime cada byte con `printf("%02X ", b)`.

```
// 1) writeInt
try (RandomAccessFile raf = new RandomAccessFile("int123.dat", "rw")) {
    raf.setLength(0);            // vaciar fichero
    raf.writeInt(123);
}
// 2) writeUTF
try (RandomAccessFile raf = new RandomAccessFile("utf123.dat", "rw")) {
    raf.setLength(0);
    raf.writeUTF("123");
}
// 3) FileWriter
try (FileWriter fw = new FileWriter("texto123.txt")) {
    fw.write("123");
}
```

| Fichero | Bytes (hex) | Tamaño | Cómo se interpreta |
|---|---|---|---|
| `int123.dat` | `00 00 00 7B` | 4 bytes | 123 = `0x7B`, en 4 bytes (el `int` ocupa siempre 4) |
| `utf123.dat` | `00 03 31 32 33` | 5 bytes | `00 03` = longitud (3 caracteres) + `31 32 33` = los códigos de '1', '2', '3' |
| `texto123.txt` | `31 32 33` | 3 bytes | Solo los caracteres, sin longitud ni relleno |

Es exactamente la tabla del anexo 1.1: `writeInt` = 4 bytes, `writeUTF("123")` = 5 bytes y `FileWriter` = 3 bytes. Los tres códigos `31 32 33` aparecen en `writeUTF` y en el texto porque son los mismos caracteres; lo que añade `writeUTF` es la longitud delante.

## 2. `Ejemplo2RandomAccess.java`: sobrescribir en cada ejecución

Es la versión 2 del ejemplo básico del tema 1.1: abre `datos.dat` en modo `"rw"` (lo crea si no existe), **lo vacía con `setLength(0)`**, escribe un registro, vuelve al inicio con `seek(0)` y lo lee.

```
try (RandomAccessFile raf = new RandomAccessFile("datos.dat", "rw")) {
    raf.setLength(0);           // borra el contenido previo
    raf.seek(0);                // al inicio (opcional)

    raf.writeInt(101);          // 4 bytes
    raf.writeUTF("Pedro");      // 2 bytes de longitud + 5 caracteres
    raf.writeDouble(8.5);       // 8 bytes

    raf.seek(0);                // el puntero vuelve al byte 0

    int id = raf.readInt();
    String nombre = raf.readUTF();
    double nota = raf.readDouble();
    System.out.println("ID: " + id + ", Nombre: " + nombre + ", Nota: " +
nota);
}
```

Salida: `ID: 101, Nombre: Pedro, Nota: 8.5`. El fichero mide **19 bytes** (4 + 7 + 8) y no crece entre ejecuciones, porque se vacía cada vez. La clave es `setLength(0)`; sin ella, cada ejecución añadiría un registro más.

## 3. El `datos.dat` subido: tres registros de 19 bytes

El `datos.dat` del aula virtual mide **57 bytes = 3 × 19**: es el resultado de ejecutar tres veces la versión anterior (sin `setLength(0)`), que añadía un registro al final cada vez. Anatomía de cada registro de 19 bytes:

| Posición (bytes) | Hex | Campo | Valor |
|---|---|---|---|
| 0–3 | `00 00 00 65` | `int` | 101 (`0x65`) |
| 4–5 | `00 05` | longitud de `writeUTF` | 5 caracteres |
| 6–10 | `50 65 64 72 6F` | caracteres | "Pedro" (P e d r o) |

| Posición (bytes) | Hex | Campo | Valor |
|---|---|---|---|
| 11–18 | `40 21 00 00 00 00 00 00` | `double` (IEEE 754) | 8.5 |

El segundo registro empieza en el byte 19 y el tercero en el 38, así que con `seek(19)` o `seek(38)` se salta directamente a cada uno. **Esto solo funciona porque los tres registros miden lo mismo** (el nombre es siempre "Pedro"): con nombres de distinta longitud los registros medirían distinto y no se podría calcular la posición (ver el ejemplo de registros de tamaño fijo del anexo 1.1).

# 4. Resumen para repasar

| Concepto | Clave |
|---|---|
| Ver los bytes reales | `Files.readAllBytes` + `printf("%02X ")` |
| `writeInt(123)` | `00 00 00 7B`: 4 bytes en binario |
| `writeUTF("123")` | `00 03 31 32 33`: 2 bytes de longitud + los caracteres |
| `writeDouble(8.5)` | `40 21 00 00 00 00 00 00`: 8 bytes IEEE 754 |
| `setLength(0)` | Vacía el fichero; evita que crezca en cada ejecución |
| Tamaño de un registro (101, "Pedro", 8.5) | 4 + (2 + 5) + 8 = 19 bytes |
| Saltar al registro n | `seek((n - 1) * tamañoRegistro)`, solo con registros de igual tamaño |